Contents

1 Introduction to Data Visualization ***3***

1.1 Types of Variables *3*

1.2 How do we describe data? *4*

1.3 Univariate Data *4*

1.3.1 Bar Chart *4*

1.3.2 Line Graph *7*

1.3.3 Pie Chart *9*

1.3.4 Dotplot *9*

1.3.5 Stemplot *11*

1.3.6 Histogram *13*

1.3.7 Cumulative Frequency Plot *16*

1.4 Bivariate Data *18*

1.4.1 Scatter Plot *18*

2 The Dot Product ***21***

2.1 Properties of the dot product *21*

2.2 Cosines and dot products *22*

2.3 Dot products in Python *26*

2.4 Work and Power *26*

3 Manufacturing ***29***

3.1 Woods and Metals Processes *29*

3.1.1 Milling *29*

3.1.2 Lathing *31*

3.2 Metal-specific Processes 32
3.2.1 Sheet Metal Processes 32
3.2.2 Casting 33
3.3 Wood-specific Processes 34
3.3.1 Bending 34
3.4 Plastic-specific Processes 35
3.4.1 Injection Molding 35
3.5 3D Printing 36
3.6 Laser Cutting and Water Jet 37

4 Sound 39
4.1 Pitch and frequency 40
4.2 Chords and harmonics 41
4.3 Making waves in Python 43
4.3.1 Making a sound file 44

5 Introduction to Graphs 47
5.1 Introduction 47
5.2 Finding Good Paths 49
5.3 Graphs in Python 50

6 Dijkstra's Algorithm 55
6.1 Algorithm Description 55
6.2 Implementation 57
6.3 Making it faster 60

7 Binary Search 63
7.1 A Naive Implementation of the Priority Queue 63
7.2 Using the Priority Queue 64
7.3 Binary Search 66
7.4 Algorithm 67

8 Other Graph Algorithms and Proofs 69
8.1 Depth-First Search 69
8.2 Bellman-Ford Algorithm 69
8.3 Prim's Algorithm 70
8.4 Breadth-First Search 70

A Answers to Exercises 71

Index 73

CHAPTER 1

Introduction to Data Visualization

It is difficult for the human mind to look at a list of numbers and identify the patterns in them, so we often use these numbers to make a picture. These pictures are called *graphs*, *charts*, or *plots*. Often, the right picture can make the meaning in the data obvious. **Data visualization** is the process of making pictures from numbers.

1.1 Types of Variables

Depending on the type of data and what you are trying to demonstrate about it, you will use different types of data visualizations. Before we get to the more complex types of data visualizations, let's look at the two types of variables we are usually looking to visualize: *categorical* and *numerical*.

Categorical or qualitative variables are those that can be put into categories. For example, the type of shoes a person wears (sneakers, sandals, etc.) is a categorical variable. The type of shoes sold is a categorical variable. The name of the salesperson (John, Jane, etc.) is also a categorical variable.

Numerical or quantitative variables are those that can be measured with numbers. For example, the number of boxes of cookies sold is a quantitative variable. The average temperature on a given day is also a quantitative variable. Quantitative variables can be further divided into *discrete* and *continuous* variables. Discrete variables are those that can only take on specific values, such as the number of pencils sold (you can't sell 3.5 pencils). Continuous variables can take on any value within a range, such as the average height of students in a class (which can be 5.5 feet, 1.65 m, etc.).

Further, we can also classify data into univariate, bivariate, and multivariate data. Univariate data has only one variable, such as the number of boxes of cookies sold. Bivariate data has two variables, such as the average temperature and total sales for a lemonade stand. Multivariate data has more than two variables, such as the average temperature, total sales, and type of ookie sold. We will only look at univariate and bivariate data!

1.2 How do we describe data?

There are different descriptive methods for representing data, and they can usually be grouped into three categories: *tabular*, *graphical*, and *numerical*.

Tabular methods include tables, which are a way of organizing data into rows and columns. They are useful for displaying raw data and making it easy to look up specific values. However, they can be difficult to interpret when dealing with large datasets or when trying to identify patterns.

Graphical methods include bar charts, line graphs, pie charts, and scatter plots. They are useful for visualizing data and making it easier to identify patterns and trends.

Numerical methods include measures of central tendency (mean, median, mode) and measures of variability (range, variance, standard deviation). They are useful for summarizing data and providing a numerical representation of the data's characteristics.

We will focus on graphical methods in this chapter, but we will also touch on tabular and numerical methods in later chapters.

1.3 Univariate Data

We will start with univariate data, which has only one variable. For example, the number of boxes of cookies sold by each salesperson is univariate data. Bar charts, line graphs, and pie charts are all good for visualizing univariate, *categorical* data. We will explore these types of graphs in the next sections.

1.3.1 Bar Chart

Bar charts are typically used to show the values of a categorical variable. Each bar represents a category, and the height of the bar represents the value of that category. Bar charts can have vertical or horizontal bars to represent this data.

Here is an example of a bar chart.

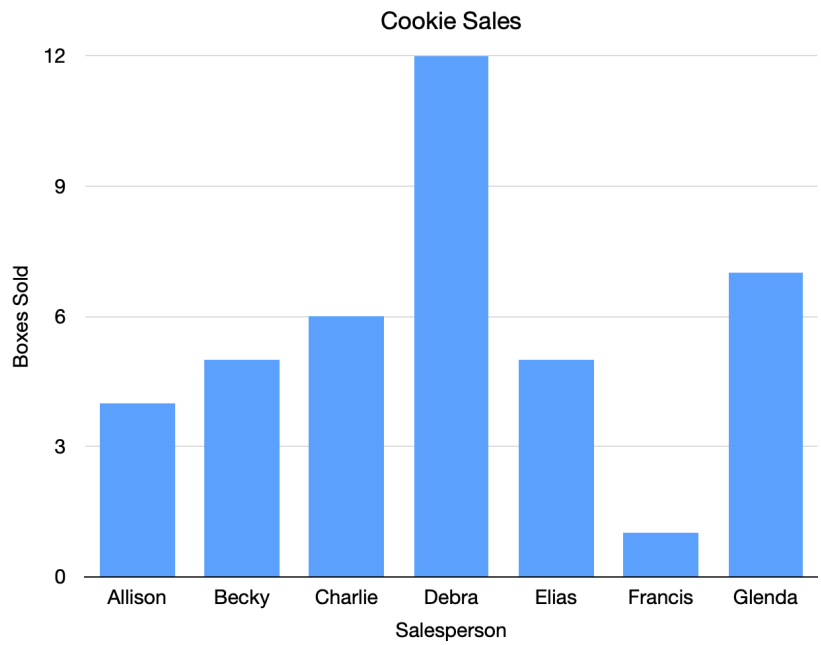


Figure 1.1: A bar chart showing cookie sales per person.

Each bar represents the cookie sales of one person. For example, Charlie has sold 6 boxes of cookies, so the bar goes over Charlie's name and reaches to the number 6.

Looking at this chart, you probably think, "Wow, Debra has sold a lot more cookies than anyone else, and Francis has sold a lot fewer."

The same data could be in a table like this:

Salesperson	Boxes Sold
Allison	4
Becky	5
Charlie	6
Debra	12
Elias	5
Francis	1
Glenda	7

A table (especially a large table) is often just a bunch of numbers. A chart helps our brains understand what the numbers mean.

Bar charts can also go horizontally.

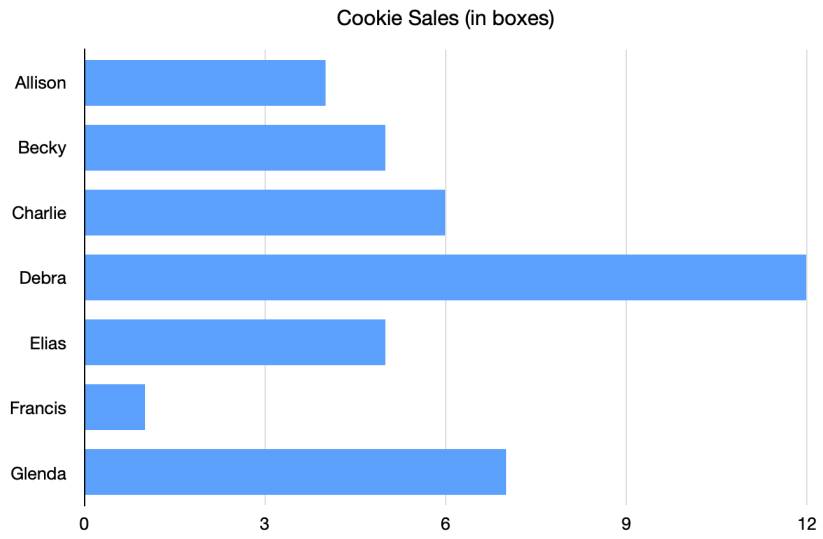


Figure 1.2: A horizontal bar chart showing the same cookie sales data.

Sometimes we use colors to explain what contributed to the number.

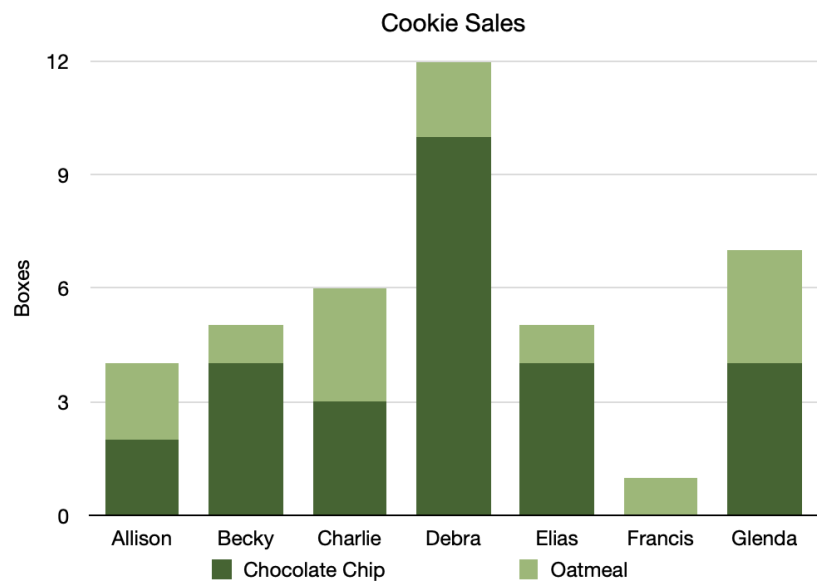


Figure 1.3: A bar chart with different colors showing types of cookies.

This tells us that Becky sold more boxes of chocolate chip cookies than boxes of oatmeal cookies.

Make Bar Graph

Go back to your compound interest spreadsheet and make a bar graph that shows both balances over time:

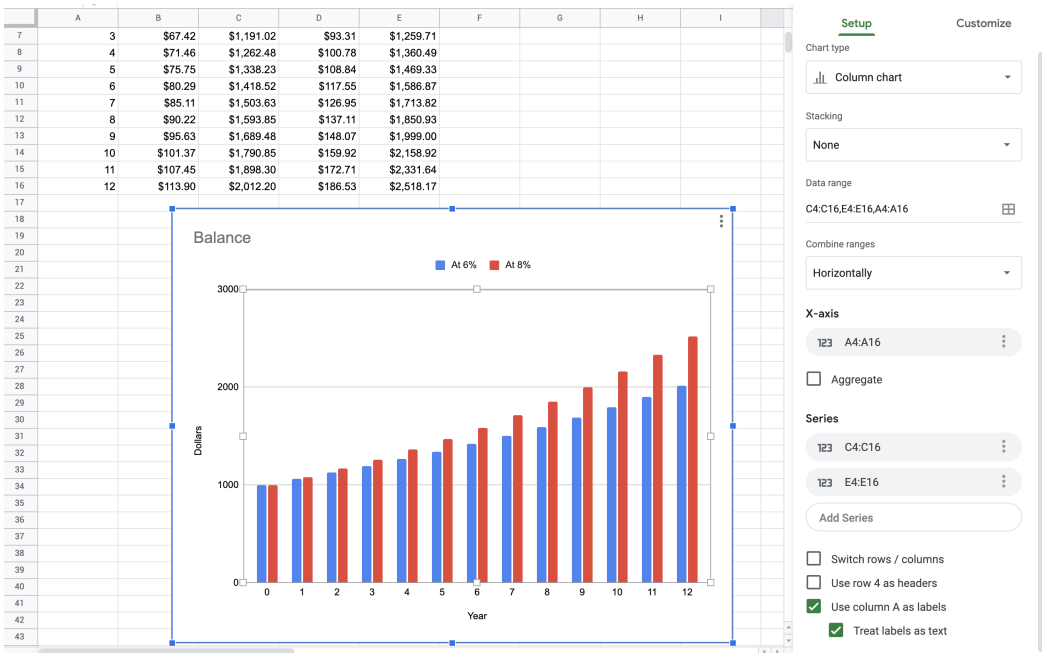


Figure 1.4: A bar graph made in google sheets showing interest.

The year column should be used as the x-axis. There are two series of data that come from C4:C16 and E4:E16. Tidy up the titles and legend as much as you like. Looking at the graph, you can see the balances start the same, but balance of the account with the larger interest rate quickly pulls away from the account with the smaller interest rate.

1.3.2 Line Graph

Here is a line graph:

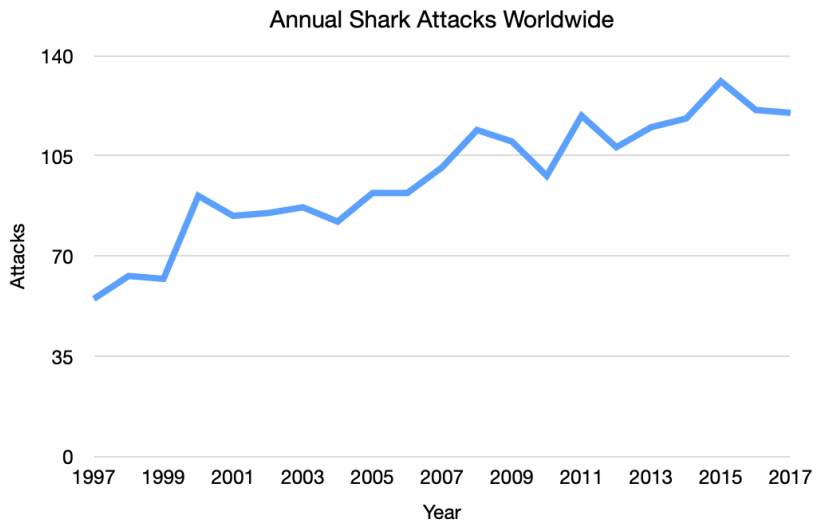


Figure 1.5: A line graph showing shark attacks per year over two decades.

These are often used to show trends over time. Here, for example, you can see that the number of shark attacks has been increasing over time.

You can have more than one line on a graph.

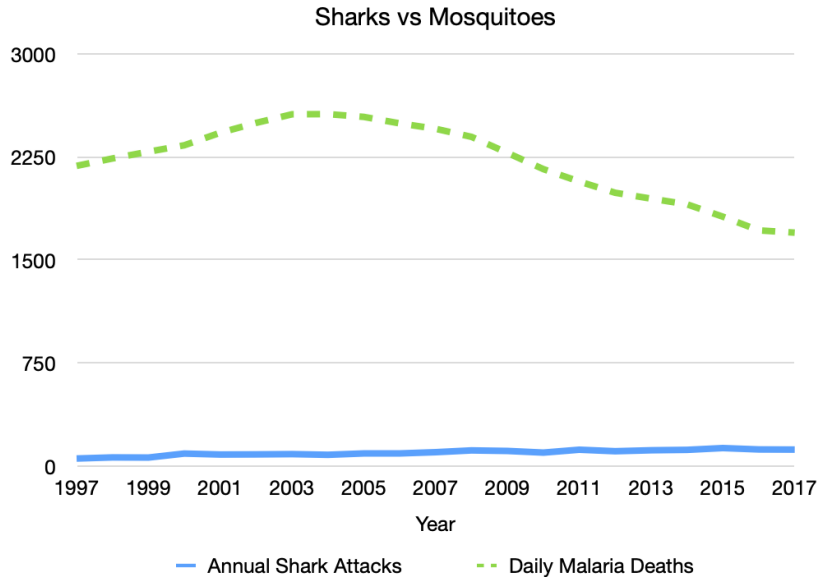


Figure 1.6: A line graph showing shark deaths versus mosquito deaths.

1.3.3 Pie Chart

You use a pie chart when you are looking at the comparative size of numbers. This is best for comparing percentages of a whole that sum to 100%. Here we can see that Nitrogen makes up 78% of the gases in the air.

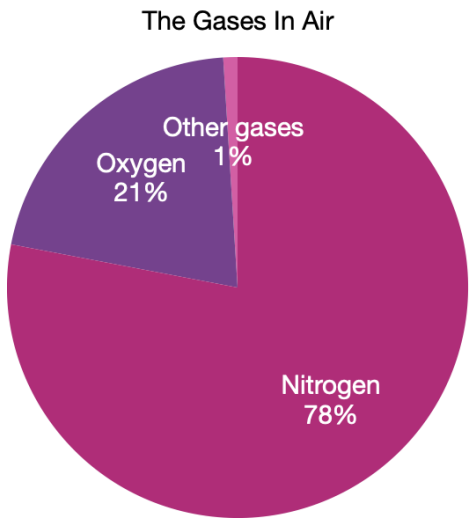


Figure 1.7: A pie chart of the various gases in the air.

When to use a Pie Chart Make a Pie Chart in Python

For *quantitative*, univariate data, we often use dotplots and stemplots to visualize smaller sets of data; and histograms, cumulative frequency plots, and boxplots to visualize larger sets of data. We will explore these types of graphs below.

1.3.4 Dotplot

A *dotplot* is one of the simplest ways to display single, quantitaive variables. Each observation is represented by a dot placed above its value on a number line. If a value appears more than once, the dots stack vertically.

Dotplots are most effective for small to moderate data sets. If the data set is too large, a dotplot can become cluttered and hard to read. In that case, a histogram or a boxplot may be a better choice.

When to Use a Dotplot

- You have 1 variable.

- The variable is quantitative.
- You want to see the distribution and the individual data values.
- The data set is not too large.

How to Make a Dotplot

1. Draw a horizontal number line (x-axis) that covers the data range.
2. Choose a scale that fits the entire range of values.
3. Place one dot above the location of each observation.
4. If multiple observations have the same value, stack the dots vertically.

Here is an example of a dotplot.

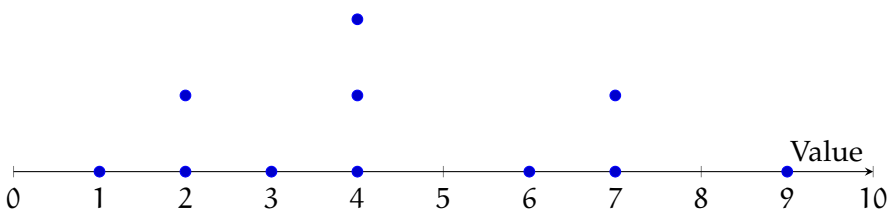


Figure 1.8: A dotplot showing a small sample distribution.

How to Read a Dotplot

Each dot shows the location of one data value. To find the value of a dot, look straight down to the number line.

A dotplot helps you describe the distribution by answering questions like:

- Spread: How spread out are the values (range and clustering)?
- Shape: Do the dots form a roughly symmetric shape, or is the distribution skewed?
- Center: Where is the distribution "balanced" (a typical value)?
- Outliers: Are there values far away from the rest?

Sometimes a dotplot shows a distribution with a long tail to one side. For example, salary data often has many values in the lower-to-middle range and a few extremely high values. This creates a right-skewed distribution.

Outliers can strongly affect numerical summaries, especially the mean.

Connecting Outliers and the Mean (Seesaw Idea) A useful way to think about the mean is as a balancing point. Imagine the dotplot as a seesaw: points far from the main cluster have more "pull." So a single extreme value can shift the mean noticeably, even if most data values stay the same.

Comparative Dotplots

A *comparative dotplot* uses the same numerical scale to compare two (or more) groups. This is useful when you want to compare distributions side-by-side.

When comparing groups, focus on differences in:

- center (typical value),
- spread,
- shape (skew/clusters),
- and outliers.

Make a Dotplot in a Spreadsheet

You can create a dotplot in Excel or Google Sheets using a scatter plot and a helper column.

1. Put your data values in column A (starting in A2). Sort them (recommended).
2. In cell B2, enter:
   `=COUNTIF($A$2:A2, A2)`
   Fill down. This creates stacking heights for repeated x-values.
3. Insert a Scatter chart using column A as x-values and column B as y-values.
4. Format the chart:
   - markers only (no lines),
   - hide the y-axis labels (y is only for stacking),
   - add titles and adjust the x-axis scale.

1.3.5 Stemplot

A *stemplot* (also called a *stem-and-leaf plot*) is an effective way to summarize **one quantitative variable** when the data set is not too large. Like a dotplot, it helps you see the overall distribution, but a stemplot has an extra advantage: it still shows the exact data values.

To make a stemplot, each observation is split into two parts:

- the stem, which is the left-most part of the number (leading digit(s))
- the leaf, which is the remaining part, usually the final digit

For example, 23 can be written as 2 | 3.

How to Make a Stemplot

1. Choose stems and leaves. Decide where to split each number into a stem and a leaf. There is no single rule for this. Your goal is to choose a reasonable number of stems.
2. Draw a vertical divider. Stems go on the left; leaves go on the right.
3. List all stems in order. Make sure the stems cover the full range of your data.
4. Write leaves for each observation. Place each leaf next to its correct stem.
5. Sort leaves. Put the leaves for each stem in increasing order.
6. Include a key. The key shows how to interpret the stems and leaves.

Choosing the Number of Stems

If you use *too few stems*, you can hide patterns because the data get lumped together. If you use *too many stems*, you can also hide patterns because the plot becomes too spread out.

A common strategy is to start with stems that represent groups of 10 (tens digits as stems), and then adjust if needed.

Sometimes a single stem represents too wide of an interval. In that case, you can split stems by repeating each stem twice:

- the first row holds leaves 0–4
- the second row holds leaves 5–9

For example, if your data run from 10 to 49, you can use stems

$$1, 1, 2, 2, 3, 3, 4, 4$$

where the first 1 represents 10–14 and the second 1 represents 15–19, and so on.

Example

Suppose these are housing costs (in dollars per month):

$$255, 259, 304, 306, 309, 317, 323, 327, 332, 334, 342, 349, 350, 351, 362, 367, 376, 384, 401, 403$$

If we use the hundreds digit as the stem and the last two digits as the leaf, we can split each stem into two rows: (top row for leaves 00–49, bottom row for leaves 50–99)

Stem	Leaves
2	55 59
2	
3	04 06 09 17 23 27 32 34 42 49
3	50 51 62 67 76 84
4	01 03
4	

Key: $3 \mid 04 = 304$ dollars per month

How to Read a Stemplot

A stemplot gives you several kinds of information at once:

- Exact values: Each leaf represents a single observation.
- Frequencies: The number of leaves in a row tells how many observations fall in that interval. For example, if a stem row has 9 leaves, that means 9 observations are in that range.
- Gaps: If a stem has no leaves, that means there are no observations in that interval.
- Shape and spread: You can see whether the values cluster in certain ranges and whether the distribution is skewed.
- Center: You can estimate a typical value by locating where the leaves cluster most.
- Outliers: Unusually high or low values may stand out because they appear far from the rest.

Just like with dotplots, you can imagine the distribution as something that could "balance" on a finger: the place it would balance is a rough sense of the center.

Make a Stem Plot in a Spreadsheet Make a Stem Plot in Python

1.3.6 Histogram

A *histogram* is one of the most popular ways to display single quantitaive variables. It is especially useful for showing patterns in larger data sets, where dotplots or stemplots can become overwhelming. A histogram is like a stemplot turned on its side: instead of listing individual values, we group values into intervals called classes or bins, and draw a bar for each bin.

Histograms are best when you care about the overall shape of a distribution (clusters, skew, gaps, outliers), not the exact value of every data point.

A histogram can be drawn using frequency (counts), relative frequency (proportions), and percent frequency (percentages). These three versions have the same shape if they use the same bins. The difference is only the y-axis scale.

How to Make a Histogram

1. Create groups from continuous data. This process is called binning.
2. Draw the x-axis and y-axis, and scale them to fit the class intervals and the frequencies (or relative frequencies or percentages).
3. Draw one bar for each class. The height of each bar equals the frequency (or relative frequency or percent) for that class.
4. Draw the bars touching with no gaps (because the data values are continuous and the bins connect).

How to Read a Histogram

- Each bar represents a single class (bin). There is one bar per class.
- The bins are placed on the x-axis in increasing order, like a number line.
- The height of a bar tells how many observations fall in that class (for a frequency histogram).
- A class with no bar means no observations fall in that interval (a gap).

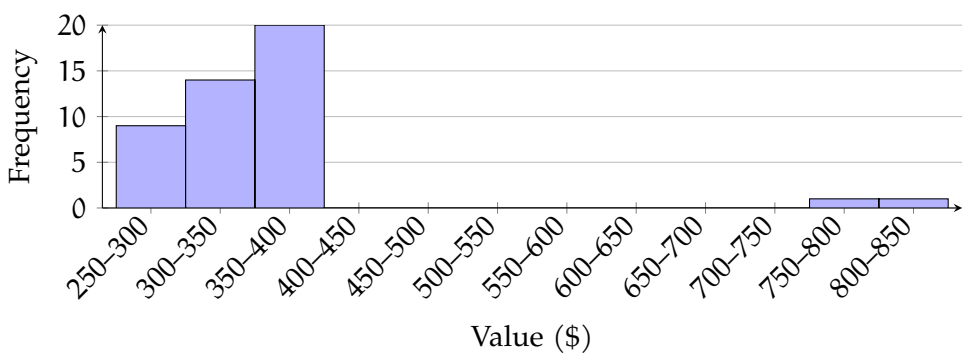


Figure 1.9: A histogram using 50-dollar bins illustrating the example counts (9, 14, 20, ... and 0,1,1 beyond $700).

Example of Interpreting Bars

Suppose a histogram uses the bins \$250–\$300, \$300–\$350, \$350–\$400, and so on.

- If the first bar (250–300) has height 9, then 9 observations are between \$250 and \$300.
- If the second and third bars (300–350 and 350–400) have heights 14 and 20, then $14 + 20 = 34$ observations are between \$300 and \$400.
- If there is no bar for 700–750, then there are 0 observations between \$700 and \$750.
- If the bars beyond 700 have heights 0, 1, and 1, then $0 + 1 + 1 = 2$ observations are at least \$700.

A histogram can help you to describe the distribution by focusing on shape, center, spread, gaps, and outliers.

Making a Histogram in a Spreadsheet

You can make a histogram in Excel or Google Sheets.

1. Put the data in one column.
2. Choose bin widths (for example, bins of width 50: 250–300, 300–350, 350–400, etc.).
3. Insert a histogram chart (or use a column chart after building a frequency table).
4. Check the bin width and the endpoints. Changing the bins can change the appearance of the distribution.

Making a Histogram in Python

Python can generate a histogram quickly and lets you change the bin width easily.

```python
import matplotlib.pyplot as plt

# Replace this list with your data
data = [250, 275, 310, 320, 350, 365, 410, 415, 430, 500, 525, 610, 720, 840]

# Choose the number of bins (try changing this!)
plt.hist(data, bins=8)

plt.title("Histogram")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```

Try changing the value of `bins`. Notice that the overall shape can look different depending

on how you group the data.

How do we compare dotplots, stemplots, and histograms?

For small data sets, dotplots and stemplots can be better because they show the exact values. In a histogram, you lose that detail because values are grouped into bins. In exchange, histograms make it easier to see the overall pattern in large data sets.

1.3.7 Cumulative Frequency Plot

A *cumulative frequency plot* (also called a *cumulative frequency chart*) shows the running total of how many data values are at or below each value (or at or below each class boundary). Instead of showing how many observations fall in each bin like a histogram, a cumulative frequency plot shows how the totals add up as you move from left to right.

Cumulative frequency plots are useful when you want to answer questions like:

- How many observations are less than or equal to a certain value?
- How many observations are greater than or equal to a certain value?
- What percent of the data fall below a certain cutoff?

How to Make a Cumulative Frequency Plot

1. Draw the x-axis and y-axis.
2. Set up bins for the data and identify the upper class boundaries (the right endpoints).
3. Compute the cumulative frequency at each upper class boundary.
   (This means: add the frequency of that class to the total from all previous bins.)
4. Plot a point at each upper class boundary with height equal to the cumulative frequency.
5. Connect the points with straight line segments.

Interpreting the shape of your plot

The steepness of the curve tells you where the data values are concentrated.

- If the curve rises quickly over a certain x-interval, many observations fall in that range.
- If the curve is almost flat, few observations fall in that range.

A cumulative frequency plot can also hint at skewness:

- **Right-skewed:** the curve increases quickly at smaller values, then levels off later.
- **Left-skewed:** the curve increases slowly at first, then rises sharply near the end.
- **Symmetric:** the curve often looks S-shaped.

Example

Suppose $n = 80$ observations were grouped into bins, and the cumulative frequency at $x = 350$ is about 23. Then:

- About 23 observations are ≤ 350.
- About $80 - 23 = 57$ observations are ≥ 350.

The plot below shows a sample cumulative frequency curve. (These coordinates are an example format; you can replace them with your own cumulative totals.)

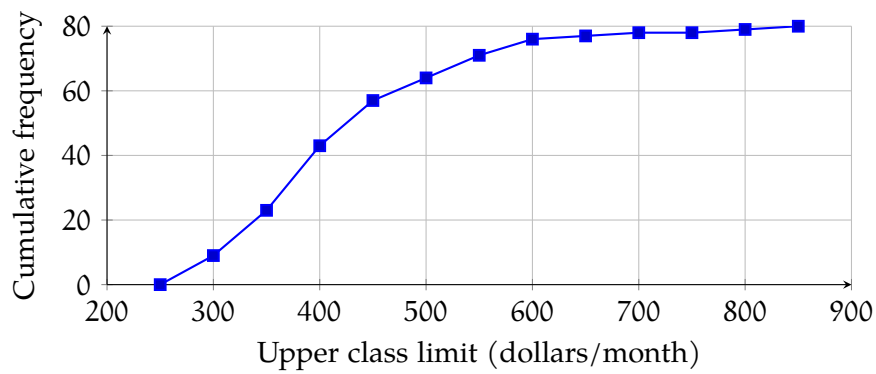


Figure 1.10: A cumulative frequency plot.

Making a Cumulative Frequency Plot in Python

This example shows the basic idea: build bins, count how many values fall in each bin, then take a cumulative sum.

```
import numpy as np
import matplotlib.pyplot as plt

# Replace with your data
data = np.array([250, 275, 310, 320, 350, 365, 410, 415, 430, 500, 525, 610, 720,
↪  840])

# Choose bin edges (upper class boundaries are the right endpoints)
```

```
bins = np.arange(250, 851, 50)  # 250, 300, ..., 850

# Frequency in each bin
counts, edges = np.histogram(data, bins=bins)

# Cumulative frequency at each upper boundary
cumulative_counts = np.cumsum(counts)
upper_bounds = edges[1:]  # right endpoints

plt.plot(upper_bounds, cumulative_counts, marker='o')
plt.title("Cumulative Frequency Plot")
plt.xlabel("Upper class boundary ($)")
plt.ylabel("Cumulative frequency")
plt.grid(True)
plt.show()
```

If you change the `bin` width, the curve may look different, but it will always increase from left to right and end at n.

Boxplot

1.4 Bivariate Data

1.4.1 Scatter Plot

Sometimes, you have a large number of data points with two values, and you are looking for a relationship between them. For example, maybe you write down the average temperature and the total sales for your lemonade stand on the 15th of every month:

Date	Avg. Temp.	Total Sales
15 January 2022	2.6º C	$183.85
15 February 2022	-4.2º C	$173.56
15 March 2022	13.3º C	$195.22
15 April 2022	26.2º C	$207.61
15 May 2022	27.5º C	$210.88
15 June 2022	31.3º C	$214.18
15 July 2022	33.5º C	$215.23
15 Aug 2022	41.7º C	$224.07
15 September 2022	20.7º C	$198.94
15 October 2022	17.2º C	$196.10
15 November 2022	1.7º C	$185.10
15 December 2022	0.2º C	$188.70

You may wonder, "Do I sell more lemonade on hotter days?"

To figure this out, you might create a scatter plot. For each day, you put a mark that represents that temperature and the sales that day:

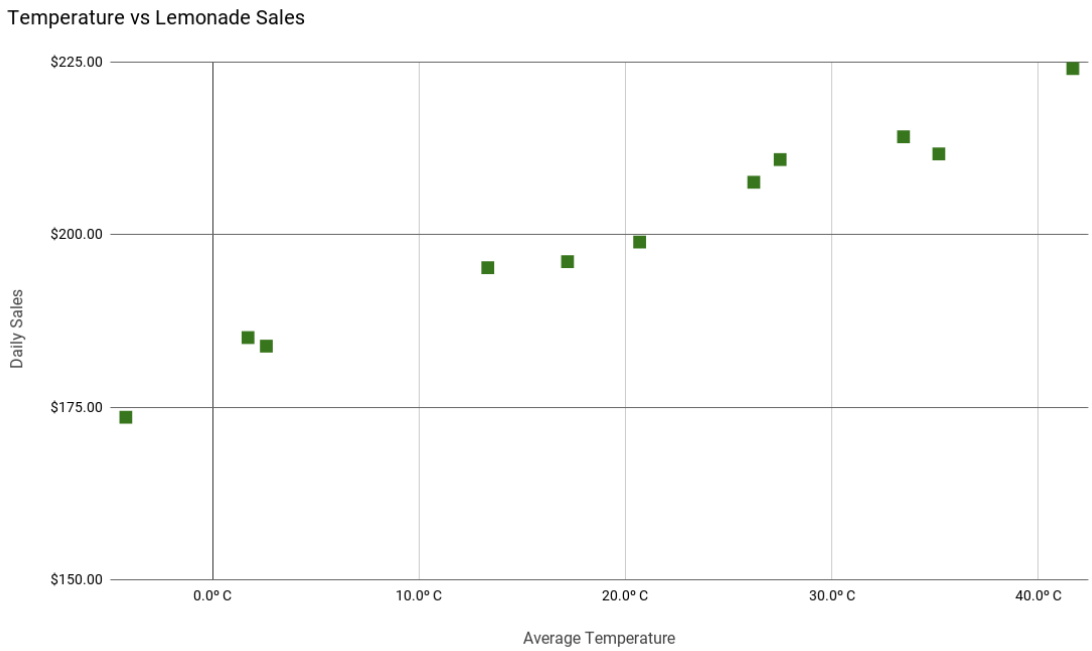


Figure 1.11: A scatterplot of temperature versus daily sales.

From this scatter plot, you can easily see that you do sell more lemonade as the temperature goes up. Drawing a *best-fit* line along the the points will give you a *correlation coefficient*. A positive correlation coefficient will give you a positively proportional relationship, while a negative coefficient will give you a inversely proportional relationship.

Making a Scatter Plot in Python

```
import matplotlib.pyplot as plt

# Avg. Temp (°C) and Total Sales ($)
temps = [2.6, -4.2, 13.3, 26.2, 27.5, 31.3, 33.5, 41.7, 20.7, 17.2, 1.7, 0.2]
sales = [183.85, 173.56, 195.22, 207.61, 210.88, 214.18, 215.23, 224.07, 198.94,
↪  196.10, 185.10, 188.70]

plt.scatter(temps, sales)
plt.title("Temperature vs. Lemonade Sales")
plt.xlabel("Average Temperature (°C)")
plt.ylabel("Total Sales ($)")
plt.grid(True)
plt.show()
```

Chapter 2

The Dot Product

If you have two vectors $\mathbf{u} = [u_1, u_2, \ldots, u_n]$ and $\mathbf{v} = [v_1, v_2, \ldots, v_n]$, we define the *dot product* $\mathbf{u} \cdot \mathbf{v}$ as

$$\mathbf{u} \cdot \mathbf{v} = (u_1 \times v_1) + (u_2 \times v_2) + \cdots + (u_n \times v_n)$$

The output of the dot product is a *scalar* quantity.

For example,

$$[2, 4, -3] \cdot [5, -1, 1] = 2 \times 5 + 4 \times -1 + -3 \times 1 = 3$$

This may not seem like a very powerful idea, but dot products are *incredibly* useful. The enormous GPUs (Graphics Processing Units) that let video games render scenes so quickly? They primarily function by computing huge numbers of dot products at mind-boggling speeds.

Exercise 1 Basic dot products

Compute the dot product of each pair of vectors:

- $[1, 2, 3]$, $[4, 5, -6]$
- $[\pi, 2\pi]$, $[2, -1]$
- $[0, 0, 0, 0]$, $[10, 10, 10, 10]$

Working Space

Answer on Page 71

2.1 Properties of the dot product

Sometimes we need an easy way to say "The vector of appropriate length is filled with zeros." We use the notation $\vec{0}$ to represent this. Then, for any vector $\mathbf{v}$, this is true:

$$\mathbf{v} \cdot \vec{0} = 0$$

The dot product is commutative:

$$\mathbf{v} \cdot \mathbf{u} = \mathbf{u} \cdot \mathbf{v}$$

The dot product of a vector with itself is its magnitude squared:

$$\mathbf{v} \cdot \mathbf{v} = |\mathbf{v}|^2$$

If you have a scalar a, then:

$$(\mathbf{v}) \cdot (a\mathbf{u}) = a(\mathbf{v} \cdot \mathbf{u})$$

So, if $\mathbf{v}$ and $\mathbf{w}$ are vectors that go in the same direction,

$$\mathbf{v} \cdot \mathbf{w} = |\mathbf{v}||\mathbf{w}|$$

If $\mathbf{v}$ and $\mathbf{w}$ are vectors that go in opposite directions,

$$\mathbf{v} \cdot \mathbf{w} = -|\mathbf{v}||\mathbf{w}|$$

If $\mathbf{v}$ and $\mathbf{w}$ are vectors that are perpendicular to each other, their dot product is zero:

$$\mathbf{v} \cdot \mathbf{w} = 0$$

2.2 Cosines and dot products

Furthermore, dot products' interaction with cosine makes them even more useful is what makes them so useful: If you have two vectors $\mathbf{v}$ and $\mathbf{u}$,

$$\mathbf{v} \cdot \mathbf{u} = |\mathbf{v}||\mathbf{u}| \cos \theta$$

where θ is the angle between them.

So, for example, if two vectors $\mathbf{v}$ and $\mathbf{u}$ are perpendicular, the angle between them is $\pi/2$. The cosine of $\pi/2$ is 0. The dot product of any two perpendicular vectors is always 0. In fact, if the dot product of two non-zero vectors is 0, the vectors *must be* perpendicular (see figure 2.1 for an example of perpendicular 2-dimensional vectors).

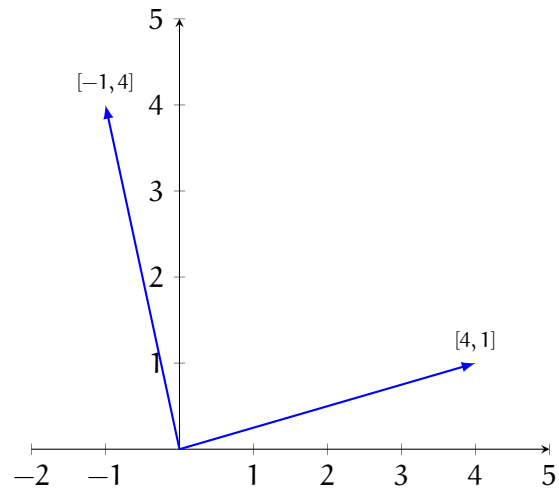


Figure 2.1: The dot product of any two perpendicular vectors is zero.

If you have two non-zero vectors $\mathbf{v}$ and $\mathbf{u}$, you can always compute the angle between them:

$$\theta = \arccos\left(\frac{\mathbf{v}\cdot\mathbf{u}}{|\mathbf{v}||\mathbf{u}|}\right)$$

Arccos is short for arccosine, or $\cos^{-}1$, and it is a function that is the inverse of cosine. Cosine takes an angle and gives back the scaled x-component of the angle. Arccosine takes the x-component of an angle and returns an angle with that x-component. However, there is a limit to what arccos can return. Let's look at cosine and its inverse, arccos (see figures 2.2 and 2.3).

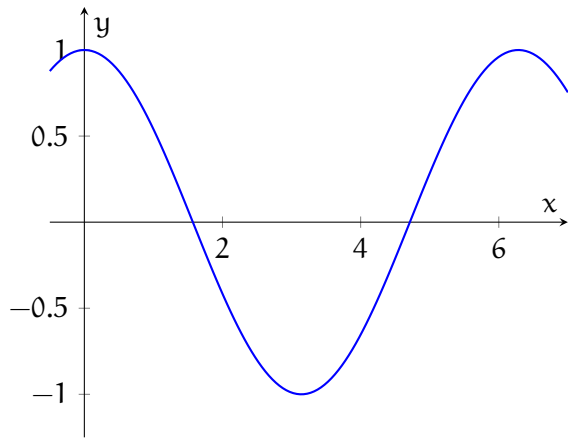


Figure 2.2: Cosine is a function: there is exactly one output for every input.

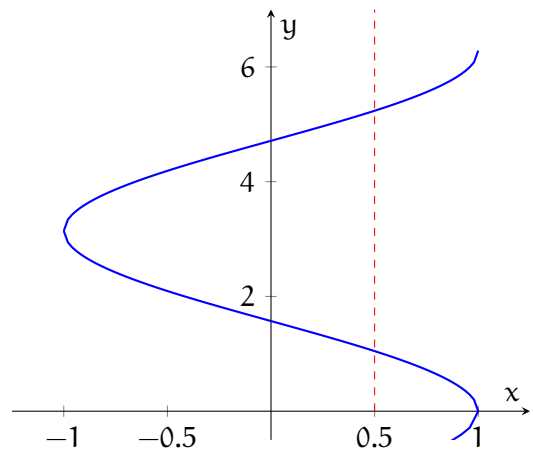


Figure 2.3: Arccos is not a function: there are many angles with the same x-component. Notice that one input value has many output values (see the red dashed line).

When you use a calculator to evaluate arccos, the calculator automatically restricts the results to between 0 and π. Let's look at an example of using the dot product to determine the angle between two vectors:

Example: What is the angle between $\mathbf{u} = \left[\sqrt{3}, 1\right]$ and $\mathbf{v} = [0, -1]$?

Solution: We know that $\mathbf{u} \cdot \mathbf{v} = |\mathbf{u}|\,|\mathbf{v}| \cos\theta$. Therefore, we also know that:

$$\cos\theta = \frac{\mathbf{u} \cdot \mathbf{v}}{|\mathbf{u}|\,|\mathbf{v}|}$$

First, let's compute the dot product:

$$\mathbf{u} \cdot \mathbf{v} = \sqrt{3} \cdot 0 + 1 \cdot -1 = -1$$

And therefore:

$$\cos\theta = \frac{-1}{|\mathbf{u}|\,|\mathbf{v}|}$$

Now, let's find the magnitudes of both vectors:

$$|\mathbf{u}| = \sqrt{\left(\sqrt{3}\right)^2 + (1)^2} = 2$$

$$|\mathbf{v}| = \sqrt{(0)^2 + (-1)^2} = 1$$

Substituting for the magnitudes, we find that:

$$\cos\theta = \frac{-1}{2 \cdot 1} = \frac{-1}{2}$$

To solve for θ, we take the arccos of both sides:

$$\arccos(\cos\theta) = \theta = \arccos\frac{-1}{2}$$

What angles have a cosine of $-1/2$? We know that $2\pi/3$, $4\pi/3$, $8\pi/3$, etc., all have a cosine of $-1/2$. Because the range of arccos is restricted to between 0 and π, our result is:

$$\theta = \arccos\frac{-1}{2} = \frac{2\pi}{3}$$

.

Therefore, the angle between $\mathbf{u}$ and $\mathbf{v}$ is $2\pi/3$ (or $120°$).

Exercise 2 Using dot products

What is the angle between these each pair of vectors:

- $[1, 0]$, $[0, 1]$
- $[3, 4]$, $[4, 3]$
- $[2, -1, 2]$, $[-1, 2, -2]$
- $[-5, 0, -1]$, $[2, 3, -4]$

Working Space

Answer on Page 71

2.3 Dot products in Python

NumPy will let you do dot products using the the symbol @. Open `first_vectors.py` and add the following to the end of the script:

```
# Take the dot product
    d = v @ u
    print("v @ u =", d)

    # Get the angle between the vectors
    a = np.arccos(d / (mv * mu))
    print(f"The angle between u and v is {a * 180 / np.pi:.2f} degrees")
```

When you run it you should get:

```
v @ u = 4
The angle between u and v is 78.55 degrees
```

2.4 Work and Power

Earlier, we mentioned that mechanical work is the product of the force you apply to something and the amount it moves. For example, if you push a train with a force of 10 newtons as it moves 5 meters, you have done 50 joules of work.

What if you try to push the train sideways? It moves down the track 5 meters, but you push it as if you were trying to derail it — perpendicular to its motion. You have done no work, because the train didn't move at all in the direction you were pushing.

Now that you know about dot products: The work you do is the dot product of the force vector you apply and the displacement vector of the train. (The displacement vector is the vector that tells how the train moved while you pushed it.)

Similarly, we mentioned that power is the product of the force you apply and the velocity of the mass you are applying it to. It is actually the dot product of the force vector and the velocity vector.

For example, if you are pushing a sled with a force of 10 newtons and it is moving 2 meters per second, but your push is 20 degrees off, you aren't transferring 20 watts of power to the sled. You are transferring $10 \times 2 \times \cos(20 \text{ degrees}) \approx 18.8$ watts of power.

Chapter 3

Manufacturing

If you try to think of any man-made object, whether it was made from woods, metals, or plastics, chances are it was produced through a manufacturing process.

Over time, these processes have been refined to be more efficient, cost-effective, and faster at producing the goods that we use on a daily basis.

New methods are also constantly being developed by engineers and scientists, and today the range of options available means that choosing the most appropriate manufacturing method involves finding the sweet spot between cost-effectiveness, yield, and time needed.

3.1 Woods and Metals Processes

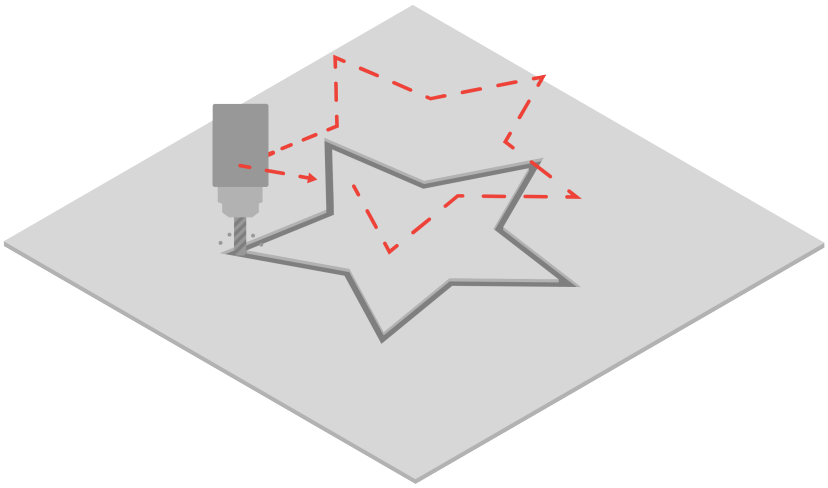
Manufacturing methods that are able to process woods and metals are typically the processes that are used to construct the vast majority of the built world around us.

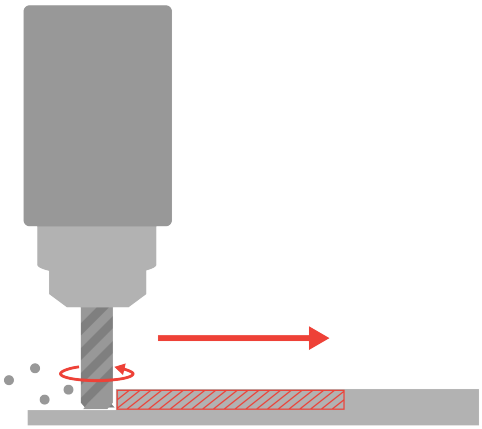
Infrastructure, transportation methods, and buildings would not exist without the advent of processes that allow us to accurately machine raw woods and metals into our desired forms.

The following sections will cover methods used to process both material types.

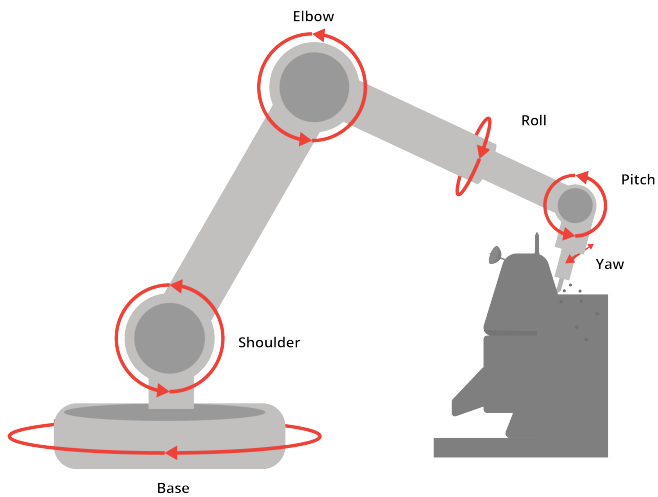
3.1.1 Milling

Mills are powerful tools that allow us to carve out complex shapes from blocks of raw material. A tool bit follows a path to remove the desired material, which makes it a *subtractive* manufacturing process. The tool bit rotates at a very high speed, which allows it to process harder materials such as woods and metals.





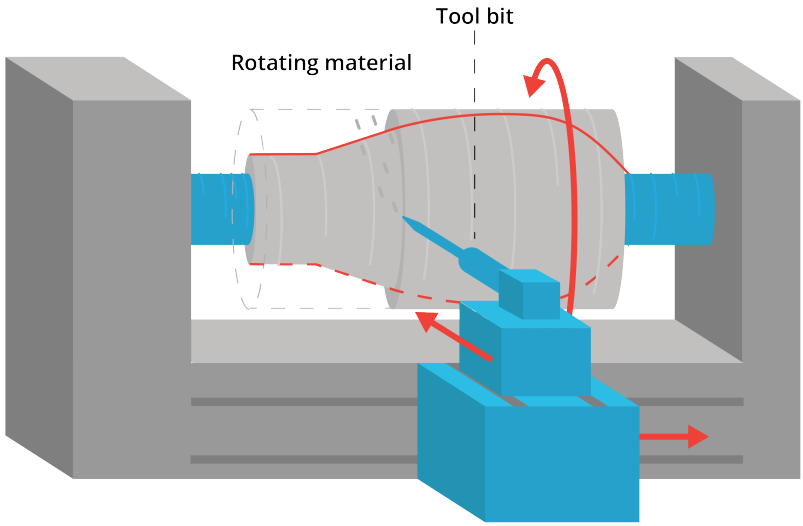
There are various types of mills, ranging from a 3-axis mill, which can cut out simple shapes in the X, Y, and Z axes, all the way to 6-axis mills, which can also rotate about those axes to create more complex curvatures.



In manufacturing settings where speed and repeatability is paramount, mills are often computer controlled. This functionality is referred to as *Computer Numerical Control*, or CNC. CNC mills are able to repeatedly follow a tool path, resulting in consistent and accurate parts.

3.1.2 Lathing

Lathes are tools that allow us to carve out complex cylindrical shapes from raw material. Like a mill, it also is a subtractive manufacturing process. However, lathes rotate the material itself at a high speed, rather than the tool bit. As the material rotates, the tool bit can be used to extract material layer by layer.



In the manufacturing industry, lathes are also often computer controlled. Alongside CNC mills, these CNC lathes are responsible for a majority of the objects that we interact with everyday.

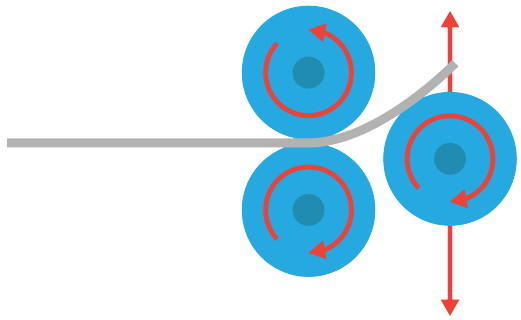
3.2 Metal-specific Processes

While mills and lathes are already able to cover the vast majority of manufacturing needs for woods and metals, there are certain processes that are specifically enabled by the unique properties of metals. More often than not, these processes leverage the malleability (ability to bend without breaking) of metals at room temperature or higher.

3.2.1 Sheet Metal Processes

While milling allows us to process blocks of metal to great effect, sometimes the application we need does not require material of such thickness.

This is where sheet metal comes in, as well as the methods we use to process it. One of the most common techniques in manufacturing is rolling, where a raw sheet is gradually rolled into the desired shape. This method is used to create many objects you may recognize, such as metal roofings, aircraft frames, and more.

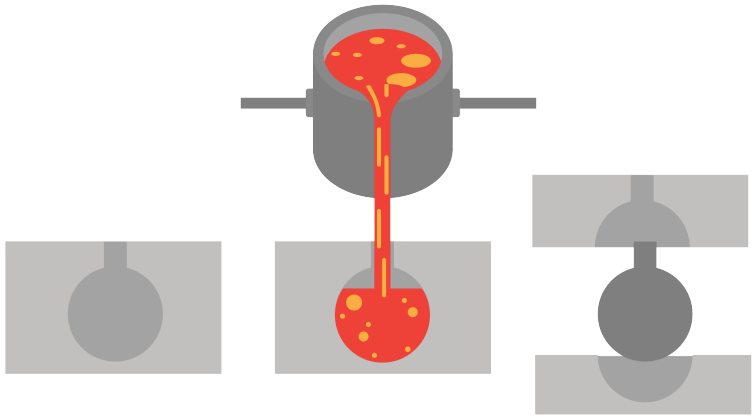


Another method is stamping, where a raw sheet is stamped into the desired shape. This allows us to create surfaces with complex geometries in an instant, and in large quantities. This method is often used for applications like the exterior panels of a car, where parts with compound curves are needed.

Lastly, there are also separate processes used to increase the strength of sheet metal parts. This falls under the category of sheet metal forming, and involves bending the edges of a part to form a reinforced flap. Almost all sheet metal parts are reinforced in this manner, as it is a relatively simple process and also helps to create a clean edge for a more finished look.

3.2.2 Casting

The last kind of metal-specific process we will cover is casting. Casting involves pouring molten metal into a mold, then letting the metal cool and set inside the mold. Smaller components with complex geometries and limited structural requirements (such as toys) are often cast, as it is an accurate and high-volume manufacturing method.



Casting also results in minimal material wastage, as it is not a subtractive manufacturing method where excess material is machined away, but rather only the specific amount of material required is poured in each time.

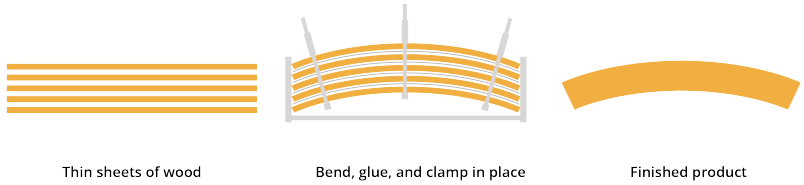
3.3 Wood-specific Processes

Similar to metals, there are also certain manufacturing processes that are enabled by the unique qualities of wood. These make use of the water content inherent in wood, and the flexibility it enables.

3.3.1 Bending

Turning raw wood into flat, workable pieces involves a variety of tools that you probably know of already, such as saws and drills. However, there are specific processes that help us create curved shapes with wood, in addition to the mill and the lathe mentioned earlier.

This is where bent lamination comes in. Bent lamination involves layering multiple thin veneers or strips of wood with adhesive, and clamping it to create the desired form while letting the glue dry. This method is often used for furniture production, enabling continuous curves in wood with tight radii.



Steam can also be used for bending, by helping soften the wood fibers to increase flexibility. Once the desired form is reached, the part can then cool down and harden. Unlike bent lamination, steam bending can be done without adhesives.

3.4 Plastic-specific Processes

Although plastics only came into prominence in the mid 20th century, they have changed manufacturing and, by extension, the world as we know it. Easily manufacturable, durable, and cost-effective, they have come to permeate almost everything we use on a daily basis.

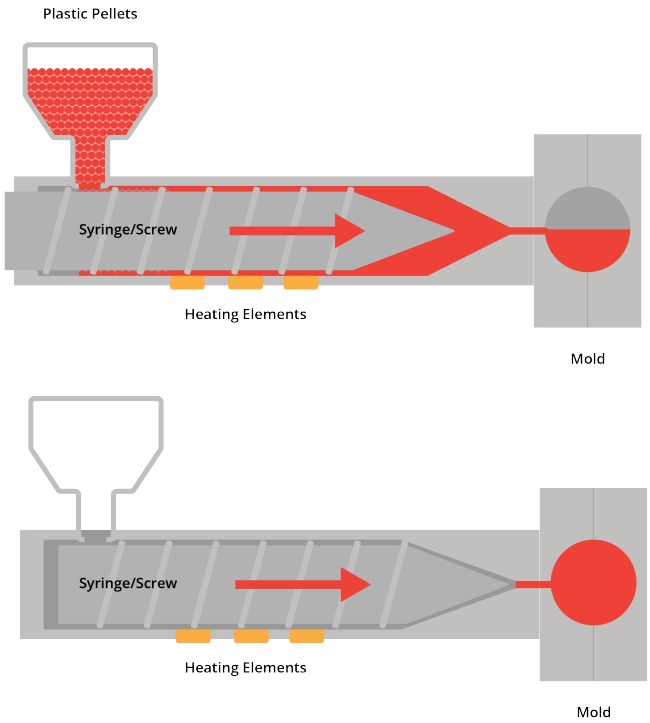
It must also be noted that these exact qualities have also resulted in the proliferation of plastics in our environment, and as such, usage of plastics should be well considered and limited. Alternative biodegradable materials are currently being trialed by scientists and would look to replicate many of the same qualities we expect from plastics, including its manufacturability.

3.4.1 Injection Molding

Injection molding is responsible for the vast majority of plastic products that you interact with on a daily basis. It is extremely quick, highly accurate, and has minimal material wastage, making it a popular and cost effective method of manufacturing plastic goods.

Similar to casting, injection molding involves injecting molten plastic into a mold, then allowing the part to cool and set inside the cavity.

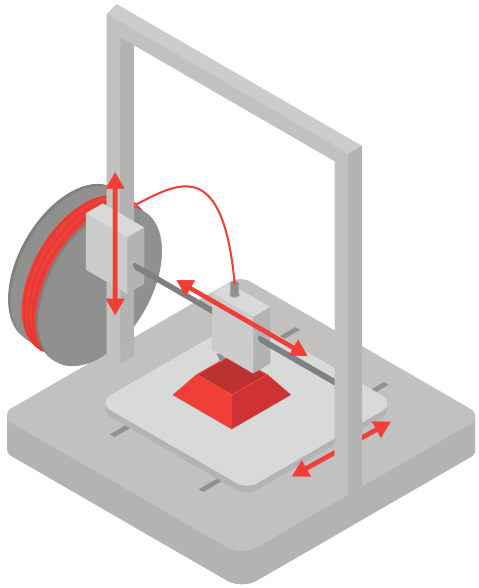
You can often tell when a part was produced through injection molding, with telltale signs such as the parting line. This is where the parts of the mold meet, forming a visible line on the surface of a part.



3.5 3D Printing

Injection molding has traditionally been the go-to technique for manufacturing plastic goods. However, new technologies result in the advent of new manufacturing methods. 3D printing is one such method, having come to prominence in the last few decades as a way to quickly prototype parts without having to create the molds needed for injection molding.

3D printing is an *additive* manufacturing process, where molten material is applied layer by layer to form the desired geometry. It allows for complex geometries, standardized batch production, and whilst the accuracy may currently lag behind traditional injection molding, it is also improving rapidly.

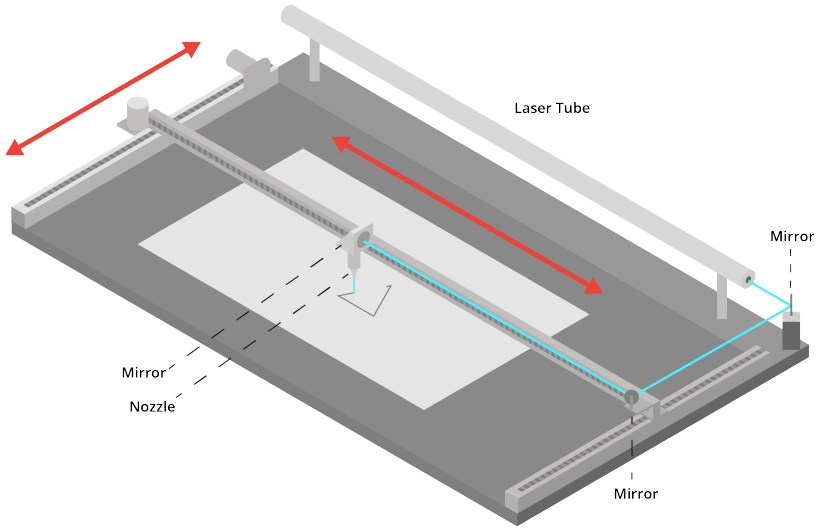


3.6 Laser Cutting and Water Jet

Similarly, another manufacturing method enabled by new technologies is laser cutting. Like 3D printing, it has come into prominence as a method to quickly prototype parts. However, it is not an additive manufacturing process.

Instead laser cutting uses a laser beam to cut and etch through sheets of plastic, however it can also be used for other materials such as fabrics and card stocks. Laser cutting is mostly limited by material thickness, and as such can only cut through thinner sheets of material.

Similarly, water jetting uses pressurized water to blast a stream of water through material (metal, wood, stone, or rubber usually). As the nozzle moves, the high pressure water traces a path throughout the material. Water jets are also limited to material thickness, as too thick of material may not easily be cut.


Laser Tube
Mirror
Mirror
Nozzle
Mirror

Chapter 4

Sound

When you set off a firecracker, it makes a sound.

Let's break that down a little more. Inside the cardboard wrapper of the firecracker, there is potassium nitrate (KNO_3), sulfur (S), and carbon(C). These are all solids. When you trigger the chemical reactions with a little heat, these atoms rearrange themselves to be potassium carbonate (K_2CO_3), potassium sulfate (K_2SO_4), carbon dioxide (CO_2), and nitrogen (N_2). Note that the last two are gasses.

The molecules of a solid are much more tightly packed than the molecules of a gas. So after the chemical reaction, the molecules expand to fill a much bigger volume. The air molecules nearby get pushed away from the firecracker. They compress the molecules beyond them, and those compress the molecules beyond them.

This compression wave radiates out as a sphere; its radius growing at about 343 meters per second ("The speed of sound").

The energy of the explosion is distributed around the surface of this sphere. As the radius increases, the energy is spread more and more thinly around. This is why the firecracker seems louder when you are closer to it. (If you set off a firecracker in a sewer pipe, the sound will travel much, much farther.)

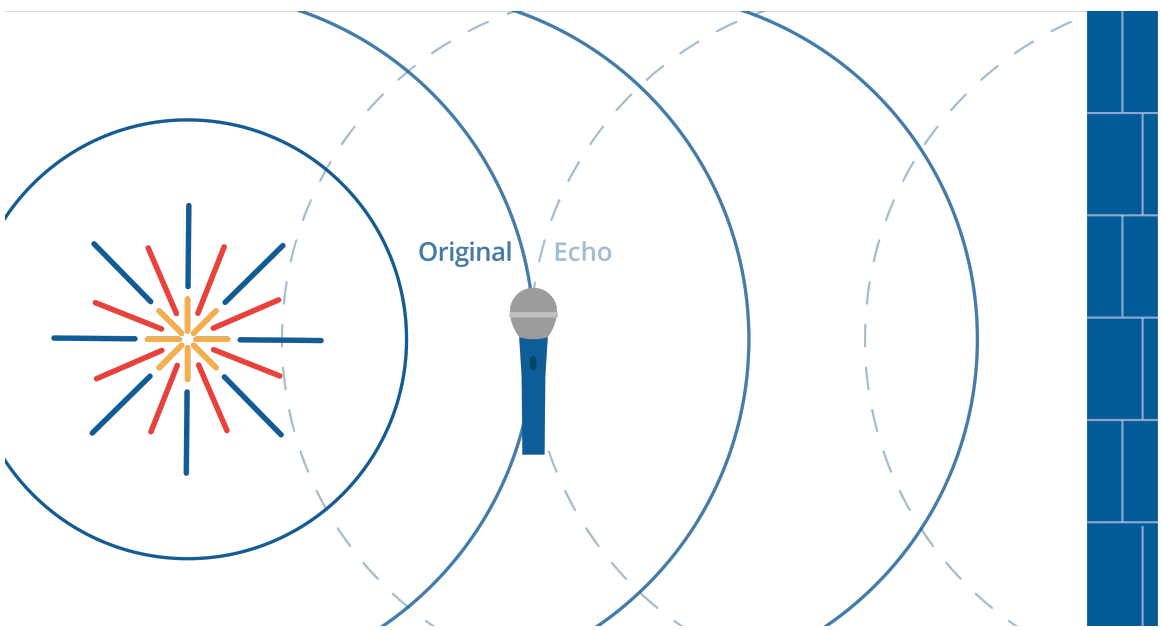


Figure 4.1: A firecracker exploding causes an initial sound wave and an echo.

This compression wave will bounce off of hard surfaces. If you set off a firecracker 50 meters from a big wall, you will hear the explosion twice. We call the second one an "echo".

The compression wave will be absorbed by soft surfaces. If you covered that wall with pillows, there would be almost no echo.

The study of how these compression waves move and bounce is called *acoustics*. Before you build a concert hall, you hire an acoustician to look at your plans and tell you how to make it sound better.

4.1 Pitch and frequency

The string on a guitar is very similar to the weighted spring example. The farther the string is displaced, the more force it feels pushing it back to equilibrium (remember the tension force?). Thus, it moves back and forth in a sine wave. (OK, it isn't a pure sine wave, but we will get to that later.)

The string is connected to the center of the boxy part of the guitar, which is pushed and pulled by the string. That creates compression waves in the air around it.

If you are in the room with the guitar, those compression waves enter your ear and push and pull your eardrum, which is attached to bones that move a fluid that tickles tiny hairs, called *cilia*, in your inner ear. This is how you hear.

We sometimes see plots of sound waveforms. The x-axis represents time. The y-axis represents the amount the air is compressed at the microphone that converted the air pressure into an electrical signal.

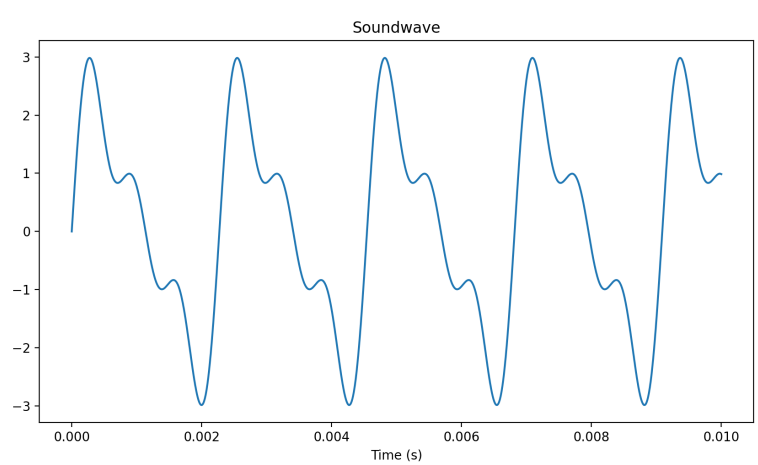


Figure 4.2: A soundwave graph over time.

If the guitar string is made tighter (by the tuning pegs) or shorter (by the guitarist's fingers on the strings), the string vibrates more times per second. We measure the number of waves per second and we call it the *frequency* of the tone. The unit for frequency is *Hertz*: cycles per second. The *period* is opposite of frequency: it is the time it takes for one cycle to complete. The unit for period is seconds per cycle.

Musicians have given the different frequencies names. If the guitarist plucks the lowest note on his guitar, it will vibrate at 82.4 Hertz. The guitarist will say "That pitch is low E." If the string is made half as long (by a finger on the 12th fret), the frequency will be twice as fast (164.8 Hertz), and the guitarist will say "That is E an octave up."

For any note, the note that has twice the frequency is one octave up. The note that has half the frequency is one octave down.

The octave is a very big jump in pitch, so musicians break it up into 12 smaller steps. If the guitarist shortens the E string by one fret, the frequency will be $82.4 \times 1.059463 \approx 87.3$ Hertz.

Shortening the string one fret always increases the frequency by a factor of 1.059463. Why?

Because $1.059463^12 = 2$. That is, if you take 12 of these hops, you end up an octave higher. This, the smallest hop in western music, is referred to as a *half step*.

Exercise 3 Notes and frequencies

Working Space

The note A near the middle of the piano, is 440Hz. The note E is 7 half steps above A. What is its frequency?

Answer on Page 71

4.2 Chords and harmonics

Of course, a guitarist seldom plays only one string at a time. Instead, they use the frets to pick a pitch for each string and strums all six strings.

Some combinations of frequencies sound better than others. We have already talked about the octave: If one string vibrates twice for each vibration of another, they sound sweet

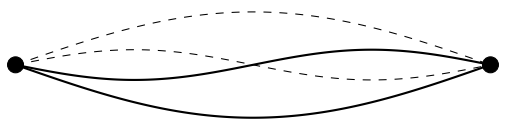
together.

Musicians speak of "the fifth". If one string vibrates three times and the other vibrates twice in the same amount of time, they sound sweet together.

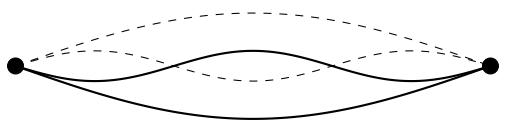
Likewise, if one string vibrates 4 times while the other vibrates 3 times, they sound sweet together. Musicians call this "the third."

Each of these different frequencies tickle different cilia in the inner ear, so you can hear all six notes at the same time when the guitarist strums their guitar.

When a string vibrates, it doesn't create a single sine wave. Yes, the string vibrates from end-to-end, and this generates a sine wave at what we call *the fundamental frequency*. However, there are also "standing waves" on the string. One of these standing waves is still at the centerpoint of the string, but everything to the left of the centerpoint is going up, while everything to the right is going down. This creates *an overtone* that is twice the frequency of the fundamental.



The next overtone has two still points — it divides the string into three parts. The outer parts are up, while the inner part is down. Its frequency is three times the fundamental frequency.



And so on. 4 times the fundamental, 5 times the fundamental, etc.

In general, tones with many overtones tend to sound bright. Tones with just the fundamental sound thin.

Humans can generally hear frequencies from 20Hz to 20,000Hz (or 20kHz). Young people tend to be able to hear very high sounds better than older people.

Dogs can generally hear sounds in the 65Hz to 45kHz range.

4.3 Making waves in Python

Let's make a sine wave and add some overtones to it. Create a file named `harmonics.py`.

```
import matplotlib.pyplot as plt
import math

# Constants: frequency and amplitude
fundamental_freq = 440.0 # A = 440 Hz
fundamental_amp = 2.0

# Up an octave
first_freq = fundamental_freq * 2.0 # Hz
first_amp = fundamental_amp * 0.5

# Up a fifth more
second_freq = fundamental_freq * 3.0 # Hz
second_amp = fundamental_amp * 0.4

# How much time to show
max_time = 0.0092 # seconds

# Calculate the values 10,000 times per second
time_step = 0.00001 # seconds

# Initialize
time = 0.0
times = []
totals = []
fundamentals = []
firsts = []
seconds = []

while time <= max_time:
    # Store the time
    times.append(time)

    # Compute value each harmonic
    fundamental = fundamental_amp * math.sin(2.0 * math.pi * fundamental_freq *
    ↪  time)
    first = first_amp * math.sin(2.0 * math.pi * first_freq * time)
    second = second_amp * math.sin(2.0 * math.pi * second_freq * time)

    # Sum them up
    total = fundamental + first + second

    # Store the values
    fundamentals.append(fundamental)
    firsts.append(first)
    seconds.append(second)
    totals.append(total)

```

```
    # Increment time
    time += time_step

# Plot the data
fig, ax = plt.subplots(2, 1)

# Show each component
ax[0].plot(times, fundamentals)
ax[0].plot(times, firsts)
ax[0].plot(times, seconds)
ax[0].legend()

# Show the totals
ax[1].plot(times, totals)
ax[1].set_xlabel("Time (s)")

plt.show()
```

When you run it, you should see a plot of all three sine waves and another plot of their sum:

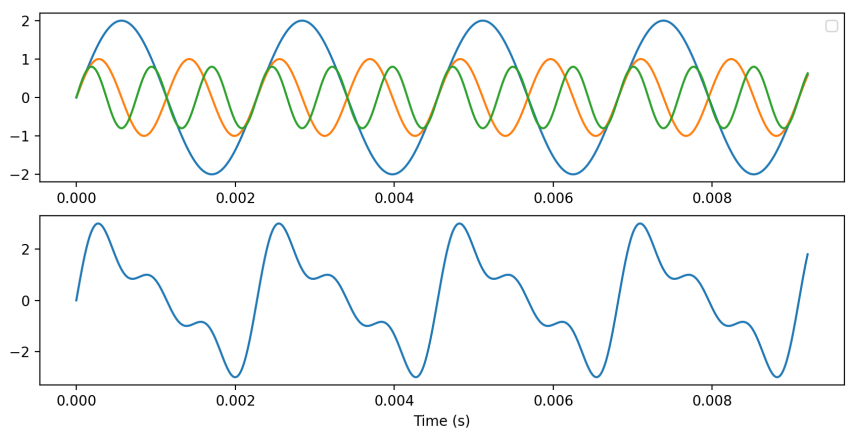


Figure 4.3: The output of haronics.py

4.3.1 Making a sound file

The graph is pretty to look at, but make let's a file that we can listen to.

The WAV audio file format is supported on pretty much any device, and a library for writing WAV files comes with Python. Let's write some sine waves and some noise into a WAV file.

Create a file called `soundmaker.py`

```python
import wave
import math
import random

# Constants
frame_rate = 16000 # samples per second
duration_per = 0.3 # seconds per sound
frequencies = [220, 440, 880, 392] # Hz
amplitudes = [20, 125]
baseline = 127 # Values will be between 0 and 255, so 127 is the baseline
samples_per = int(frame_rate * duration_per) # number of samples per sound

# Open a file
wave_writer = wave.open('sound.wav', 'wb')

# Not stereo, just one channel
wave_writer.setnchannels(1)

# 1 byte audio means everything is in the range 0 to 255
wave_writer.setsampwidth(1)

# Set the frame rate
wave_writer.setframerate(frame_rate)

# Loop over the amplitudes and frequencies
for amplitude in amplitudes:
    for frequency in frequencies:
        time = 0.0
        # Write a sine wave
        for sample in range(samples_per):
            s = baseline + int(amplitude * math.sin(2.0 * math.pi * frequency *
            ↪  time))
            wave_writer.writeframes(bytes([s]))
            time += 1.0 / frame_rate

        # Write some noise after each sine wave
        for sample in range(samples_per):
            s = baseline + random.randint(0, 15)
            wave_writer.writeframes(bytes([s]))

# Close the file
wave_writer.close()
```

When you run it, it should create a sound file with several tones of different frequencies and volumes. Each tone should be followed by some noise.

CHAPTER 5

Introduction to Graphs

5.1 Introduction

Some data is easier to work with if we imagine it as a set of connections. When we say *graph* in this chapter, we are referring to set of data defined by a collection of *nodes* (you may also hear vertex) and connected by *edges*. For example, on some social networks, each user can follow any number of other users. We can think of each user as a node, and the edge points from the user who follows to the user they follow:

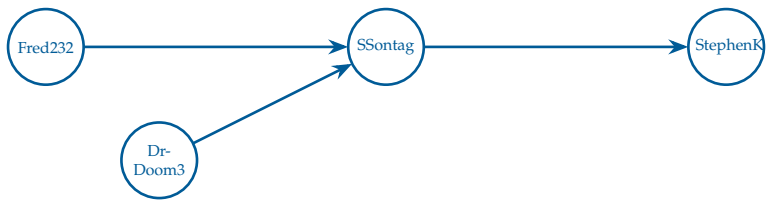


Figure 5.1: A graph showing four users and three "following" relationships.

Following is a *directed* relationship: Fred232 follows SSontag, but SSontag doesn't follow Fred232. So we would say that this is a *directed graph* with four nodes and three edges.

There are also undirected graphs. For example, you can imagine a graph that represents big data lines between cities. All the big data lines allow communications in both directions, like two-way roads:

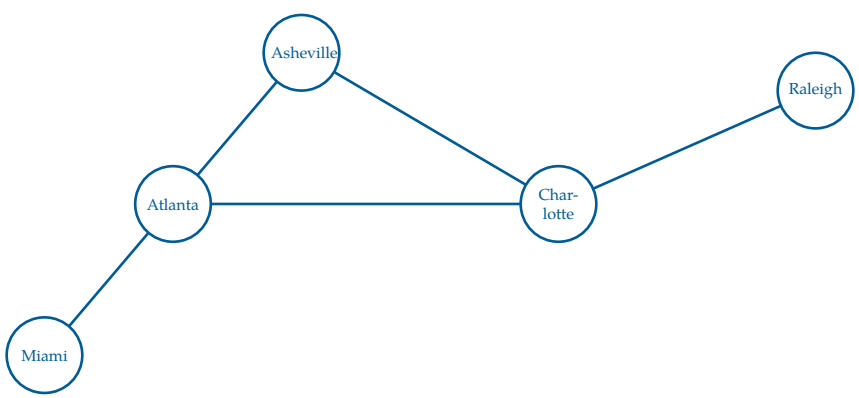


Figure 5.2: A set of cities, represented by nodes, on the east coast of the United States connected by roads, represented by edges.

The arrows are gone; if data can flow from Miami to Raleigh, then data can flow from Raleigh to Miama. A linked connection of edges between two or more nodes is called a *path*. Notice how Atlanta and Charlotte have 3 edges; we would say that Atlanta and Charlotte each have a *degree* of 3.

There is a whole branch of mathematics called *Graph Theory* that studies the properties of graphs. Here are two questions that mathematicians might ask about this graph:

- What is the smallest number of edges that we would need to follow to get from Miami to Raleigh?
- Does the graph have any paths where you could end up where you started?

That second question asks if there is a *cycle* in the graph. This graph has one cycle: Atlanta $\rightarrow$ Asheville $\rightarrow$ Charlotte $\rightarrow$ Atlanta.

Some graphs are *connected*: You can get from one node to any other node by following edges. Is this graph connected?

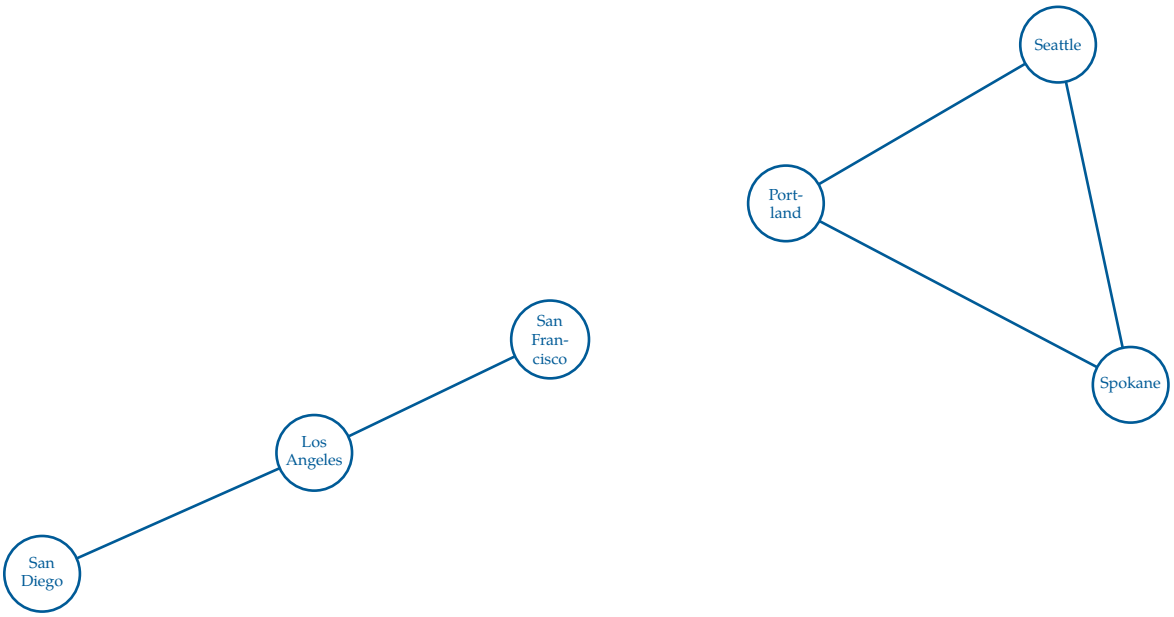


Figure 5.3: A set of cities (nodes) that are not all connected. Each set of *connected* cities forms a *connected component*.

This graph is *not* connected! You can't follow edges from San Diego to Seattle. However, you could identify two *connected components*, *subgraphs* of a graph that remain connected.

- San Diego, Los Angeles, San Francisco
- Portland, Seattle, Spokane

Connectivity is defined as, for all pairs of nodes in that graph, there exists a path between them. A subgraph is much simpler than a connected component, it's only requirement is to contain a subset of at least one node and, optionally, edges to go between nodes. Note that you cannot have a subgraph that contains a single node and an edge, because the edge would not be able to connect to anything else.

In graph data, the nodes and edges often have attributes. For example, a node representing a city might have a name and a population. An edge representing a data line might have a bandwidth (bits per second) and a latency (how many nanoseconds between when you put a bit into the pipe and when it comes out the other end). This is how we define a graph.

5.2 Finding Good Paths

For many problems, we are trying to find the best path from one node to another. If all the edges are the same, this usually means finding the path that requires walking the fewest edges.

Sometimes the edges have a cost attribute. For example, you might want to find the cheapest way to ship a container from New York City to Long Beach, California. In this case the nodes are train depots. Each train line between the depots has a cost. What is the cheapest path?

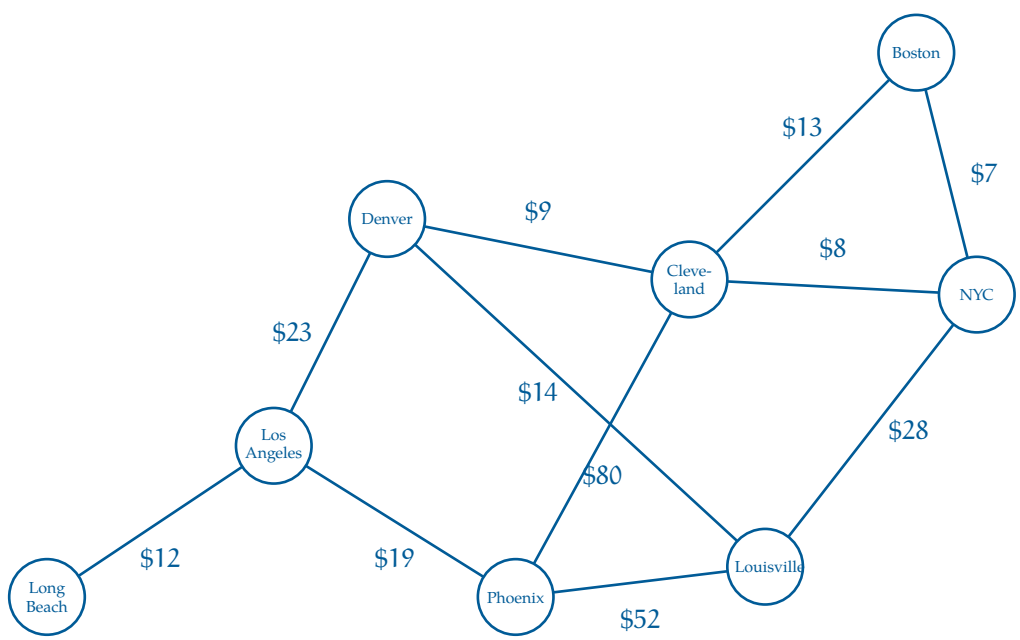


Figure 5.4: A graph of interconnected cities where each edge has a respective cost.

When edges have costs like this, we call the *weighted edges*, which forms a *weighted graph*. In weighted graphs, the *degree* of a node is split into the *in-degree* and *out-degree* of nodes. The in-degree identifies the incoming edges that originate from other nodes, and out-degree is the amount of edges going out of that node.

The graphs that you see here are really small, so finding efficient paths isn't difficult — you could just try all of them! However, in many computer programs, we are working with *millions* of nodes and edges. Efficient graph algorithms are *really* important.

5.3 Graphs in Python

There are many ways to represent graph data. There are database systems that are specifically designed to hold and analyze graph data. Not surprisingly, these are called *Graph Databases*.

You may also hear about *adjacency lists*, which store, for each node, the list of nodes it is connected to. Each direct connection from one node to another is called a *neighbor*. Simply, adjacency lists store lists of neighbors.

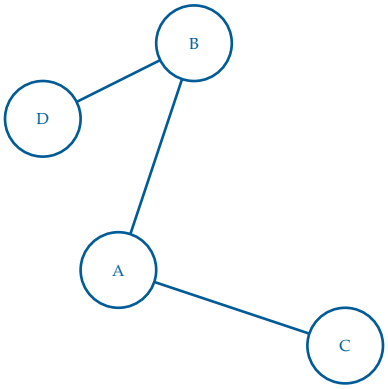


Figure 5.5: A simple (undirected unweighted) graph used to make an adjacency list.

As an adjacency list, this would look like:

- A: [B, C]
- B: [A, D]
- C: [A]
- D: [B]

Exercise 4 Directed Adjacency Lists

Working Space

Consider the concept of adjacency lists discussed above. How would the storage method change if the edges were to have weights?

Answer on Page 71

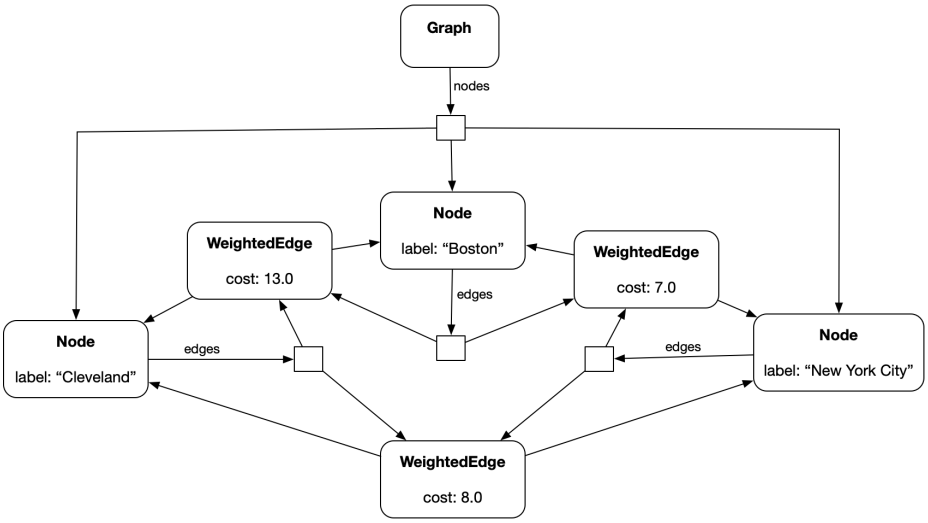
For simplicity, you are going to write Python classes that will let you represent an undirected graph with weighted edges, like the shipping problem above.

(Naturally, things would look a little different if the graph were directed or the edges were unweighted, but this is a good starting place.)

Create a file called `graph.py`. This will hold the code for your `Node` and `WeightedEdge` classes. We will also create a `Graph` class that will just hold onto the list of its nodes.

- A `Node` will have a label string and a list of edges that touch it.
- A `WeightedEdge` will have a cost and two nodes: `node_a` and `node_b`.
- A `Graph` will have a list of nodes.

Here is what the object diagram would look like if you had only three cities:



Put this code into `graph.py`

```
class Node:
  def __init__(self, label):
    self.label = label
    self.edges = []

  def __repr__(self):
    return f"(node:{self.label}, edges:{len(self.edges)})"

class WeightedEdge:
  def __init__(self, cost, node_a, node_b):
    self.cost = cost
    self.node_a = node_a
    node_a.edges.append(self)
    self.node_b = node_b
    node_b.edges.append(self)

def other_end(self, node_from):
    if self.node_a == node_from:
        return self.node_b
    if self.node_b == node_from:
        return self.node_a
    raise ValueError("node is not an endpoint of this edge")

class Graph:
  def __init__(self):
    self.nodes = []

  def add_node(self, new_node):
    self.nodes.append(new_node)

  def __repr__(self):
    return f"(Graph:{self.nodes})"
```

This is a graph class that handles undirected weighted graphs. Now, let's create some instances of `Node` and `WeightedEdge` and wire them together. Create another file in the same directory called `cities.py`. Put in this code:

```
import graph

# Create an empty graph
network = graph.Graph()

# Create city nodes and add to graph
long_beach = graph.Node("Long Beach")
network.add_node(long_beach)
los_angeles = graph.Node("Los Angeles")
network.add_node(los_angeles)
denver = graph.Node("Denver")
network.add_node(denver)
phoenix = graph.Node("Phoenix")
network.add_node(phoenix)
louisville = graph.Node("Louisville")
network.add_node(louisville)
cleveland = graph.Node("Cleveland")
network.add_node(cleveland)
boston = graph.Node("Boston")
network.add_node(boston)
nyc = graph.Node("New York City")
network.add_node(nyc)

# Create edges
graph.WeightedEdge(12, long_beach, los_angeles)
graph.WeightedEdge(23.0, los_angeles, denver)
graph.WeightedEdge(19, los_angeles, phoenix)
graph.WeightedEdge(52, phoenix, louisville)
graph.WeightedEdge(14, denver, louisville)
graph.WeightedEdge(80, phoenix, cleveland)
graph.WeightedEdge(9, denver, cleveland)
graph.WeightedEdge(8, cleveland, nyc)
graph.WeightedEdge(28, louisville, nyc)
graph.WeightedEdge(7, nyc, boston)
graph.WeightedEdge(13, cleveland, boston)

print(network)
```

Run it:

```
python3 cities.py
```

You should see some rather unexciting output:

```
(Graph:[(node:Long Beach, edges:1), (node:Los Angeles, edges:3), (node:Denver,
↪  edges:3),
(node:Phoenix, edges:3), (node:Louisville, edges:3), (node:Cleveland, edges:4),
(node:Boston, edges:2), (node:New York City, edges:3)])
```

But do not worry; we will make it more exciting in the next chapter!

CHAPTER 6

Dijkstra's Algorithm

Edsger W. Dijkstra was a great Dutch computer scientist. He came up with an algorithm for finding the cheapest path through a graph with weighted edges. Today it is known as *Dijkstra's Algorithm*. It is used in a wide variety of common problems, and is also both simple and elegant.

6.1 Algorithm Description

You are going to mark each node with how much it would cost to get there from some chosen origin node. For example, if you are shipping a container from Long Beach, you will mark each city with the cost of getting the container to that city.

You start by marking the price for Long Beach to zero (the container is already there). You then mark each adjacent city with the cost on the edge. Next, you declare Long Beach to be "visited".

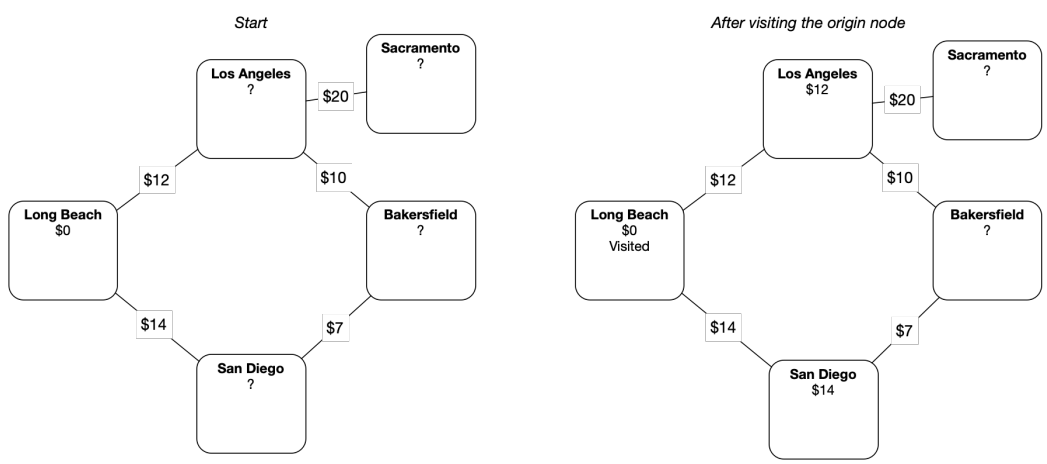


Figure 6.1: A map of weighted cities.

After that, you find the cheapest of the unvisited nodes. In this case, Los Angeles is cheaper than San Diego, so that is the node you will visit next.

You mark all of the unvisited nodes adjacent to Los Angeles, with the price to ship it to Los Angeles plus the cost of shipping the container from Los Angeles to that city. Note that Bakersfield is marked with $22.

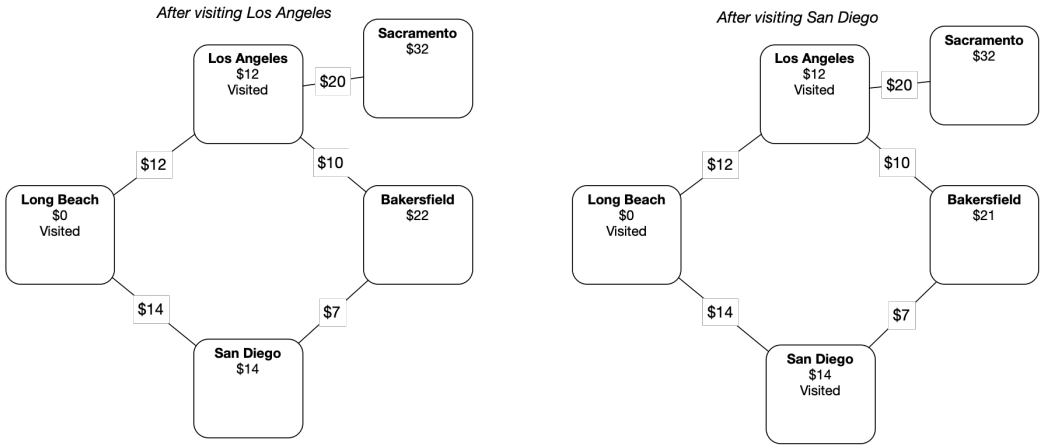


Figure 6.2: Marking each node with the cost to get to that city from the origin.

Now, the cheapest unvisited node is San Diego, so you mark its neighbors with the cost to ship to San Diego, plus the price to ship from San Diego to the neighbor.

Notice that Bakersfield is already labeled with $22 from a route through Los Angeles. But the price would be $21 if you shipped it to Bakersfield via San Diego. Because the new route is cheaper, you change the price to the lower value.

(What does it mean that a node is "visited"? If a node is marked visited, it is marked with a price that won't get any smaller.)

You continue visiting the cheapest unvisited node until all the nodes have been visited. Then you know every node has been marked with its lowest price.

In a big graph, each node may be marked several times in this process — each time with a lower price from a cheaper router.

Of course, once you have the price, you will ask, "What is the route that gets me that price?" So, we will also mark each node with the neighbor from which it would receive the shipment — the previous node. This is easy to do as we execute the algorithm.

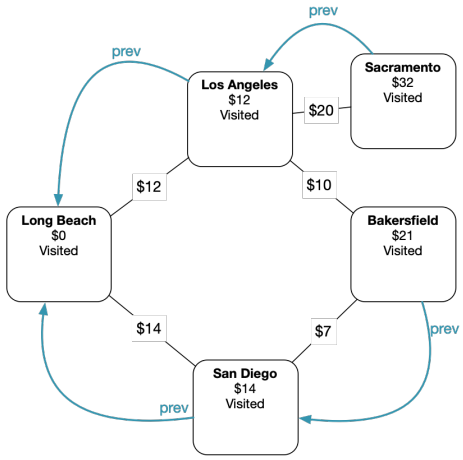


Figure 6.3: Marking each previous node.

Now, to figure out the cheapest route from San Diego to Bakersfield, we start at the destination and follow the `prev` pointer back through San Diego, then to Long Beach.

6.2 Implementation

We do not actually want to sully our graph objects with the three additional pieces of information we need:

- The current minimal cost from the origin node. This is usually called the `dist`, from "distance".
- The neighbor who gives us the current minimal cost. This is usually called `prev`, from "previous".
- Whether the city is visited or now.

So, we will keep them in collections external to the graph.

For example, to keep track of the `dist`, we will have a `dist` dictionary. Each node will be a key, the current minimal cost will be the value. If the node hasn't received even a first cost, we will put in infinity as the cost.

(After the algorithm is run, if the cost of a node is still infinity, that means that it cannot be reached from the origin node.)

We will also have a `prev` dictionary. The final node will be the key, and its previous neighbor will be the value.

Finally, the graph has a list of all the nodes, so we can just keep a set of the unvisited nodes.

Add a method to the `Graph` class that implements Dijkstra's algorithm:

```
    def cost_from_node(self, origin_node):
        # Cost of cheapest path from origin node discovered so far
        # Initially the origin is zero and all the other are infinity
        dist = {k: math.inf for k in self.nodes}
        dist[origin_node] = 0.0

        # The previous city on that cheapest path
        prev = {}

        # All the nodes start as unvisited
        unvisited = set(self.nodes)

        # While there are still unvisited nodes
        while unvisited:

            # Find unvisited node with lowest cost
            min_cost = math.inf
            for u in unvisited:
                if dist[u] < min_cost:
                    current_node = u
                    min_cost = dist[u]

            # If none are less than inf, we are done
            # This happens in graphs that are not connected
            if min_cost == math.inf:
                return (dist, prev)

            # Remove the lowest cost node from the unvisited list
            unvisited.remove(current_node)

            # Update all the unvisited neighbors
            for edge in current_node.edges:

                # What node is at the other end of this edge?
                v = edge.other_end(current_node)

                # Visited nodes are already minimized, skip them
                if v not in unvisited:
                    continue

                # Is this a shorter route?
                alt = dist[current_node] + edge.cost
                if alt < dist[v]:

                    # Update the distance and prev dicts
                    dist[v] = alt
                    prev[v] = current_node
```

```

        return (dist, prev)
```

Append some code to your `cities.py` that tests this method:

```
(cost_from_long_beach, prev) = network.cost_from_node(long_beach)
print(f"\nMinimum costs from Long Beach = {cost_from_long_beach}")
print(f"\nLast city before = {prev}")

nyc_cost = cost_from_long_beach[nyc]

if nyc_cost < math.inf:
    print(f"\n*** Total cost from Long Beach to NYC: ${nyc_cost:.2f} ***")
else:
    print("You can't get to NYC from Long Beach")
```

When you run it, you should get a list of how much it costs to ship a container to each city from Long Beach:

```
Minimum costs from Long Beach = {(node:Long Beach, edges:1): 0.0,
(node:Los Angeles, edges:3): 12.0, (node:Denver, edges:3): 35.0,
(node:Phoenix, edges:3): 31.0, (node:Louisville, edges:3): 49.0,
(node:Cleveland, edges:4): 44.0, (node:Boston, edges:2): 57.0,
(node:New York City, edges:3): 52.0}
```

You will also get a collection of node pairs. What are these? For each node, you get the node that you would pass through on the cheapest route from Long Beach:

```
Last city before = {(node:Los Angeles, edges:3):(node:Long Beach, edges:1),
(node:Denver, edges:3):(node:Los Angeles, edges:3),
(node:Phoenix, edges:3):(node:Los Angeles, edges:3),
(node:Louisville, edges:3):(node:Denver, edges:3),
(node:Cleveland, edges:4):(node:Denver, edges:3),
(node:New York City, edges:3):(node:Cleveland, edges:4),
(node:Boston, edges:2): (node:Cleveland, edges:4)}
```

Your users will not want to read this; give them the shortest path as a list. Add a function to `graph.py` that turns the `prev` table into a path of nodes that lead from the origin to the destination:

```
def shortest_path(prev, destination):

    # Include the destination in the path
    path = [destination]
```

```
    current_node = destination

    # Keep stepping backward in the path
    while current_node in prev:

        # What node should come before the current node?
        previous_node = prev[current_node]

        # Insert it at the start of the list
        path.insert(0, previous_node)
        current_node = previous_node

    return path
```

Test that out:

```
if nyc_cost < math.inf:
    print(f"*** Total cost from Long Beach to NYC: ${nyc_cost:.2f} ***")

    path_to_nyc = graph.shortest_path(prev, nyc)
    print(f"*** Cheapest path from Long Beach to NYC: {path_to_nyc ***")}
else:
    print("You can't get to NYC from Long Beach")
```

This should look like this:

```
*** Cheapest path from Long Beach to NYC: [(node:Long Beach, edges:1),
(node:Los Angeles, edges:3), (node:Denver, edges:3), (node:Cleveland, edges:4),
(node:New York City, edges:3)] ***
```

6.3 Making it faster

On really big networks, doing a full Dijkstra's algorithm would take too long; luckily, there are many methods for getting similar results quickly. When you ask for directions from Google Maps, it does not do a full Dijkstra's Algorithm for every possible route — it would just take too long.

However, there is a way to speed up this implementation. Look at this snippet:

```
            # Find unvisited node with lowest cost
            min_cost = math.inf
            for u in unvisited:
                if dist[u] < min_cost:
                    current_node = u
```

```
                    min_cost = dist[u]
```

We are scanning through the list of all unvisited nodes, one by one, looking for the one with the lowest cost. If we kept this list sorted by cost, then the next one to visit would always be the first one in the list. This is done with a *priority queue* – a list that keeps itself sorted by some priority number – in this case the cost. In python, the standard priority queue is `heapq`.

(So why didn't I implement this using `heapq`? For Dijkstra's Algorithm, the nodes' priority – the current cost – changes as we find cheaper routes. `heapq` doesn't handle the changing priority very gracefully.)

In the next chapter, you will make a priority queue class that will work in this case.

CHAPTER 7

Binary Search

As mentioned in the last chapter, you are going to make a priority queue for use with Dijkstra's Algorithm. Using it will look like this:

```
import kpqueue

myqueue = pqueue.PriorityQueue()
myqueue.add(long_beach, 0) # Inserts first city and its cost
myqueue.add(san_diego, 14) # Puts San Diego after Long Beach
myqueue.add(los_angeles,12) # Inserts LA between Long Beach and San Diego
current_city = myqueue.pop() # Returns first city (Long Beach) and removes it
```

Now, if an item gets a new priority, we need to remove it and reinsert it in the new spot.

```
myqueue.add(city_a, 16) # Puts it last in the queue
myqueue.update(city_a, 16, 13) # Moves it to between LA and San Diego
```

7.1 A Naive Implementation of the Priority Queue

Create a file called `kpqueue.py`. Let's do a simple implementation that stores the priority and the data as tuple. And we will keep it sorted by the priority. If two tuples have the same priority, we will sort by the data.

Type this in to `kpqueue.py`:

```
class PriorityQueue:
    def __init__(self):
        self.list = []

    # Return and remove the first item
    def pop(self):
        if len(self.list) > 0:
            return self.list.pop(0)
        else:
            return None

    def __len__(self):
        return len(self.list)
```

```

    def update(self, value, old_priority, new_priority):
        old_pair = (old_priority, value)
        self.list.remove(old_pair)
        self.add(value, new_priority)

    def add(self, value, priority):
        pair = (priority, value)
        # Add it at the end
        self.list.append(pair)
        # Resort the list
        self.list.sort()
```

This will work fine, but it could be much more efficient:

- Every time we add a single element, we resort the whole list.
- The function `remove` is searching the list sequentially for the item to delete.

In a minute, we will revisit these inefficiencies and make them better.

7.2 Using the Priority Queue

We are going to change `graph.py` to use the priority queue. While we are doing so, why don't we also shrink the memory footprint of our program a bit.

Notice that as the algorithm is running, each node is in one of three states:

- Unseen: In the earlier implementation, these were the nodes with `math.inf` as their cost.
- Seen, but not finalized: These are "unvisited", but don't have `math.inf` as their cost.
- Finalized: These are the "visited" nodes — we know that their cost won't decrease any more.

We can shrink the memory foot print by not putting the unseen into the dist dictionary at all. And instead of a separate set for "unvisited", what if we moved finalized nodes and their distances into a separate dictionary?

Rewrite the `cost_from_node` function in `graph.py`:

```
            # Visited nodes are already minimized, skip them
              if v in finalized_dist:
                  continue
```

```

                # What is the cost to this neighbor?
                alt = current_node_cost + edge.cost

                # Is this the first time I am seeing the node?
                if v not in seen_dist:

                    # Insert into the seen_dict, prev, and priority queue
                    seen_dist[v] = alt
                    prev[v] = current_node
                    pqueue.add(v, alt)

                else: # v has been seen. Is this a cheaper route?
                    old_dist = seen_dist[v]
                    if alt < old_dist:
                        # Update the seen_dict, prev, and priority queue
                        seen_dist[v] = alt
                        prev[v] = current_node
                        pqueue.update(v, old_dist, alt)

        return (finalized_dist, prev)
```

This should be have exactly the same result, except for the unreachable nodes. If you have a graph that is not connected, there will be nodes that cannot be reached from the origin. In the old version, these had a cost of `math.inf`. Now, they will just not be in the dictionary at all. So, change `cities.py` to deal with this:

```
if nyc in cost_from_long_beach:
    nyc_cost = cost_from_long_beach[nyc]
    print(f"\n*** Total cost from Long Beach to NYC: ${nyc_cost:.2f} ***")

    path_to_nyc = graph.shortest_path(prev, nyc)
    print(f"\n*** Cheapest path from Long Beach to NYC: {path_to_nyc} ***")
else:
    print("You can't get to NYC from Long Beach")
```

If you run `cities.py` now, it should behave exactly like the old version.

However, there is a bug. It will rear its head if two cities with the same cost are in the priority queue together. Change `cities.py` so that Denver and Phoenix have the same cost:

```
graph.WeightedEdge(12, long_beach, los_angeles)
graph.WeightedEdge(19, los_angeles, denver)
graph.WeightedEdge(19, los_angeles, phoenix)
```

Now try running it. You should get an error:

```
TypeError: '<' not supported between instances of 'Node' and 'Node'
```

What happened? The `loc_for_pair` method is comparing tuples made up of a `float` and a `Node`. The `float` comes first in the tuple, so that is compared first. However, if the two tuples have the same priority, it then compares nodes.

The error statement says, "Nodes don't have a less-than method; I don't know how to compare them."

Each `Node` lives at an address in memory. You can get that address as a number using the `id` function. The ID is unique and constant over the life of the object. It is a rather arbitrary ordering, but it will work for this problem. Add a method to your `Node` class:

```
    # Nodes will be ordered by their location in memory
    def __lt__(self, other):
        return id(self) < id(other)
```

Fixed.

Now, let's make the priority queue more efficient.

7.3 Binary Search

The phone company in every town used to print a thing called a phone book. The names and phone numbers were arranged alphabetically. As you might imagine, these books often had more than a thousand pages.

If you were looking for "John Jeffers", you would not start at the first page and read sequentially until you reached his name. You would open the book in the middle, and see a name like "Mac Miller", then think "Jeffers comes before Miller". You would then split the pages in your left hand in half and see a name like "Hester Hamburg" and think "Jeffers comes after Hamburg". You would then split the pages in your right hand, and continue on like this until you found the page with "John Jeffers" on it. That is binary search.

Binary Search is a search algorithm that finds the position of a target value within a sorted array. The binary search algorithm works by repeatedly dividing the search interval in half. If the target value is equal to the middle element of the array, the position is returned. If the target value is less than or greater than the middle element, the search continues in the lower or upper half of the array, respectively.

7.4 Algorithm

The binary search algorithm can be described as follows:

1. If the array is empty, the search is unsuccessful, so return "Not Found".
2. Otherwise, compare the target value to the middle element of the array.
3. If the target value matches the middle element, return the middle index.
4. If the target value is less than the middle element, repeat the search with the lower half of the array.
5. If the target value is greater than the middle element, repeat the search with the upper half of the array.
6. Repeat steps 2-5 until the target value is found or the array is exhausted.

```
class PriorityQueue:
    def __init__(self):
        self.list = []

    # Return and remove the first item
    def pop(self):
        if len(self.list) > 0:
            return self.list.pop(0)
        else:
            return None

    def __len__(self):
        return len(self.list)

    def add(self, value, priority):
        pair = (priority, value)
        i = self.loc_for_pair(pair)
        self.list.insert(i, pair)

    def update(self, value, old_priority, new_priority):
        old_pair = (old_priority, value)
        i = self.loc_for_pair(old_pair)
        del self.list[i]
        self.add(value, new_priority)

    def loc_for_pair(self, pair):
        # The range where it could be is [lower, upper)
        # Start with the whole list
        lower = 0
        upper = len(self.list)

        while upper > lower:
            next_split = (upper + lower) // 2
            v = self.list[next_split]
```

```
            if pair < v:  # pair is to the left
                upper = next_split
            elif pair > v:  # pair is to the right
                lower = next_split + 1
            else: # Found pair!
                return next_split
        return lower
```

If you try running it now, it should work perfectly.

You now have a graph class that will find the cheapest path quickly, even if it has thousands of nodes with thousands of edges.

CHAPTER 8

Other Graph Algorithms and Proofs

Now that you are familiar with Dijkstra's algorithm for finding the shortest path in a graph, you are well-equipped to understand more graph algorithms. This document will discuss two other important algorithms: *Depth-First Search* (*DFS*) and the Bellman-Ford algorithm.

8.1 Depth-First Search

Depth-First Search (DFS) is an algorithm for traversing or searching tree or graph data structures. DFS uses a stack (or sometimes recursion, which uses the system stack implicitly) to explore the graph in a depthward motion until it hits a node with no unvisited adjacent nodes, then it backtracks.

The procedure is as follows:

1. Push the root node into the stack.
2. Pop a node from the stack, and mark it as visited.
3. Push all unvisited adjacent nodes into the stack.
4. Repeat steps 2 and 3 until the stack is empty.

DFS is particularly useful for solving problems like connected-component detection in graphs and maze-solving.

8.2 Bellman-Ford Algorithm

The Bellman-Ford Algorithm is another shortest path algorithm like Dijkstra's. However, unlike Dijkstra's Algorithm, Bellman-Ford can handle graphs with negative weight edges.

The algorithm works as follows:

1. Assign a tentative distance value for every node: set it to zero for our initial node

and to infinity for all other nodes.

2. For each edge (u, v) with weight w, if the current distance to v is greater than the distance to u plus w, update the distance to v to be the distance to u plus w.
3. Repeat the previous step $|V| - 1$ times, where $|V|$ is the number of vertices in the graph.
4. After the above steps, if you can still find a shorter path, there exists a negative cycle.

If the graph does not contain a negative cycle reachable from the source, the shortest paths are well-defined, and Bellman-Ford will correctly calculate them. If a negative cycle is reachable, no solution exists, but Bellman-Ford will detect it.

8.3 Prim's Algorithm

Prim's Algorithm is FIXME https://github.com/KontinuaFoundation/sequence/issues/315

8.4 Breadth-First Search

Appendix A

Answers to Exercises

Answer to Exercise 1 (on page 21)

- $[1, 2, 3] \cdot [4, 5, -6] = 4 + 10 - 18 = -4$
- $[\pi, 2\pi] \cdot [2, -1] = 2\pi - 2\pi = 0$
- $[0, 0, 0, 0] \cdot [10, 10, 10, 10] = 0 + 0 + 0 + 0 = 0$

Answer to Exercise 2 (on page 26)

- $[1, 0] \cdot [0, 1] = 0$. The angle must be $\pi/2$.
- $[3, 4] \cdot [4, 3] = 24$. $|[3, 4]||[4, 3]| \cos(\theta) = 24$. $\cos(\theta) = \frac{24}{(5)(5)}$. $\theta = \arccos(\frac{24}{25}) \approx 0.284$ radians.
- $[2, -1, 2] \cdot [-1, 2, -2] = 4 - 2 - 4 = -2$. $|[2, -1, 2]| = \sqrt{4 + 1 + 4} = \sqrt{9} = 3$. $|[-1, 2, -2]| = \sqrt{1 + 4 + 4} = \sqrt{9} = 3$. $3(3) \cos\theta = -2$. $\theta = \arccos(-2/9) \approx 1.795$ radians.
- $[-5, 0, -1] \cdot [2, 3, -4] = -10 + 0 + 4 = -6$. $|[-5, 0, 1]| = \sqrt{25 + 0 + 1} = \sqrt{26}$. $|[2, 3, -4]| = \sqrt{4 + 9 + 16} = \sqrt{29}$. $\sqrt{26}(\sqrt{29}) \cos\theta = -6$. $\theta = \arccos\left(\frac{-6}{\sqrt{26}\sqrt{29}}\right) \approx 1.791$ radians.

Answer to Exercise 3 (on page 41)

A is 440 Hz. Each half-step is a multiplication by $\sqrt[12]{2} = 1.059463094359295$ So the frequency of E is $(440)(2^{7/12}) = 659.255113825739859$

Answer to Exercise 4 (on page 51)

In a weighted graph, each node would have to store a set of neighbors and their respective weights. The best way to do this would be a tuple. For example,

- A: [(B, 4), (C, 3)]

- B: [(A, 4), (D, 8)]
- C: [(A, 3)]
- D: [(B, 8)]

Index

acoustics, 40
additive, 36
adjacency lists, 50
an overtone, 42

bar chart, 4
Bellman-Ford algorithm, 69
best-fit, 19
Binary Search, 66

cilia, 40
comparative dotplot, 11
Computer Numerical Control, 31
connected, 48
connected component, 48
connected components, 48
correlation coefficient, 19
cycle, 48

degree, 48
Depth-First Search, 69
depth-first search, 69
DFS, 69
Dijkstra's Algorithm, 55
Dijkstra, Edsger, 55
directed, 47
directed graph, 47
dot product, 21
dotplot, 9

edges, 47

frequency, 41

graph, 47
    connected, 48
    directed, 47
    undirected, 47
Graph Databases, 50
Graph Theory, 48

half step, 41
Hertz, 41

in-degree, 50

line graph, 7

neighbor, 50
nodes, 47

out-degree, 50
outlier, 10

path, 48
period, 41
pie chart, 9
power, 27
Priority Queue, 63
priority queue, 61

scatter plot, 18
subgraphs, 48
subtractive, 29

the fundamental frequency, 42

vectors
    angle between, 23

weighted edges, 50
weighted graph, 50
work, 27